

Solidum FEM

Plataforma de Análisis por Elementos Finitos en Python

Sigla: FF-MU

Manual de Usuario

Sintaxis del archivo YAML, catálogos de componentes,
ejemplos completos y workflow de ejecución

Autor: Jaime Retama Velasco

Facultad de Estudios Superiores Aragón
Universidad Nacional Autónoma de México

23 de septiembre de 2026

Índice general

Sobre este manual	v
1. Introducción y Configuración del Entorno	1
1.1. Filosofía del Proyecto	1
1.1.1. Modelo Data-Driven	1
1.2. Requisitos del Sistema	1
1.2.1. Dependencias	1
1.2.2. Dependencias opcionales	2
1.3. Herramientas Externas Recomendadas	2
1.4. Ejecución de un Análisis	2
2. Arquitectura del Programa	3
2.1. Separación Elemento $\leftrightarrow$ Material	3
2.2. Auto-Registro de Componentes	3
2.3. Validación Temprana del Input	4
3. Archivo de Entrada YAML	5
3.1. Geometría: <code>nodes</code> y <code>elements</code>	5
3.2. Importación de Malla Gmsh: <code>mesh</code>	6
3.2.1. Mapeo Avanzado: <code>mesh_physical_groups</code>	6
3.3. Materiales	6
3.4. Condiciones de Frontera	7
3.4.1. Por nodo: <code>boundary_conditions_by_node</code>	7
3.4.2. Por coordenada: <code>boundary_conditions_by_coord</code>	7
3.4.3. Por grupo físico: <code>boundary_conditions_by_group</code>	7
3.4.4. Restricciones multipunto (MPC): <code>linear_constraints</code>	7
3.5. Cargas Puntuales	8
3.6. Fuerzas de Cuerpo	8
3.6.1. Forma general: <code>body_force</code>	9
3.6.2. Caso particular — peso propio: <code>gravity + density</code>	9
3.6.3. Detalles transversales	9
3.7. Solver	10
3.7.1. Bloque <code>convergence</code>	10
3.8. Salida: <code>output</code>	11

4. Catálogo de Elementos Finitos	13
4.1. Elementos 1D	13
4.1.1. Armaduras: <code>Truss2D</code> / <code>Truss2DCorot</code> / <code>Truss3D</code> / <code>Truss3DCorot</code>	13
4.1.2. Cables: <code>Cable2DCorot</code> / <code>Cable3DCorot</code>	14
4.1.3. Marcos 2D: <code>Frame2DEuler</code> / <code>Frame2DTimoshenko</code> / <code>Frame2DEulerCorot</code>	14
4.1.4. Marco 3D: <code>Frame3D</code>	15
4.2. Elementos 2D	16
4.2.1. Cuadrilátero Bilineal: <code>Quad4</code>	16
4.2.2. Triángulo de Deformación Constante: <code>Tri3</code>	16
4.2.3. Cuadrilátero Serendípito de Orden 2: <code>Quad8</code>	17
4.2.4. Cuadrilátero Lagrangiano de Orden 2: <code>Quad9</code>	17
4.2.5. Triángulo Cuadrático Completo P_2 : <code>Tri6</code>	17
4.3. Elementos 2D con discontinuidad embebida	18
4.3.1. Triángulo CST con Discontinuidad Interior: <code>CST_Embedded2D</code>	18
4.4. Elementos 3D	19
4.4.1. Hexaedro Trilineal: <code>Hex8</code>	19
4.4.2. Tetraedro Lineal: <code>Tet4</code>	19
4.4.3. Hexaedro Serendípito de Orden 2: <code>Hex20</code>	19
4.4.4. Hexaedro Lagrangiano Triquadrático: <code>Hex27</code>	20
4.4.5. Tetraedro Cuadrático: <code>Tet10</code>	20
5. Catálogo de Modelos Constitutivos	21
5.1. Elasticidad Lineal	21
5.1.1. <code>Elastic1D</code> — elasticidad lineal axial	21
5.1.2. <code>Elastic2D</code> — elasticidad lineal isótropa 2D	21
5.2. Plasticidad	21
5.2.1. <code>Elastoplastic1D</code> — plasticidad J2 axial con endurecimiento isótropo	21
5.2.2. <code>VonMises2D</code> — plasticidad J2 plana con endurecimiento isótropo	22
5.2.3. <code>DruckerPrager2D</code> — plasticidad friccional cohesivo-friccional 2D	23
5.3. Daño Mecánico	24
5.3.1. <code>IsotropicDamage1D</code> — daño isótropo 1D con softening exponencial	24
5.3.2. <code>IsotropicDamage2D</code> — daño isótropo 2D	24
5.4. Elasticidad Unilateral (Cable)	25
5.4.1. <code>CableMaterial1D</code> — cable lineal en tensión, nulo en compresión	25
5.5. Materiales 3D	25
5.5.1. <code>Elastic3D</code> — elástico lineal isótropo 3D	26
5.5.2. <code>VonMises3D</code> — plasticidad J2 3D con endurecimiento isótropo lineal	26
5.5.3. <code>DruckerPrager3D</code> — plasticidad friccional 3D	26
5.5.4. <code>IsotropicDamage3D</code> — daño isótropo 3D con softening exponencial	26
5.6. Materiales Cohesivos (salto de desplazamientos)	27
5.6.1. <code>CohesiveDamageIsotropic</code> — daño cohesivo Modo-I	27

6. Esquemas de Solución	28
6.1. LinearSolver — solución directa lineal	28
6.2. NonlinearSolver — Newton-Raphson incremental con paso adaptativo	28
6.3. ArcLengthSolver — longitud de arco cilíndrico (Crisfield)	29
6.4. DissipationArcLengthSolver — arc-length por disipación de energía	30
7. Análisis Dinámico	31
7.1. ModalSolver — análisis modal por autovalores generalizados	31
7.2. NewmarkSolver — transitorio lineal Newmark- β	32
7.3. NewtonNewmarkSolver — transitorio no lineal Newton-Newmark	33
7.4. HHTSolver y NewtonHHTSolver — variantes Hilber-Hughes-Taylor	34
7.5. CentralDifferenceSolver — explícito por diferencias centradas (ADR 0009 fase 5)	35
7.6. HarmonicSolver — respuesta forzada armónica en frecuencia (ADR 0009 fase 6)	35
7.7. ResponseSpectrumSolver — análisis sísmico por combinación modal (ADR 0009 fase 7)	36
7.8. Mass lumping (ADR 0009 fase 2)	38
8. Análisis Térmico	39
8.1. Formulación	39
8.2. Material: ThermalConduction	39
8.3. Elementos: Quad4Thermal y Hex8Thermal	40
8.4. Condiciones de frontera	41
8.4.1. Dirichlet — temperatura impuesta	41
8.4.2. Neumann — flujo prescrito	41
8.4.3. Fuente volumétrica	41
8.5. Régimen estacionario	42
8.6. Régimen transitorio: ThetaMethodSolver	42
8.6.1. Elección de θ	42
8.6.2. Elección de Δt	43
8.6.3. Capacidad lumped por defecto	43
8.6.4. Resultado	43
8.7. Ejemplo completo: pared plana en régimen transitorio	44
8.8. Limitaciones actuales	45
9. Post-Procesamiento y Visualización	46
9.1. Formato VTU (VTK XML)	46
9.1.1. Datos Nodales (Point Data)	46
9.1.2. Datos de Elemento (Cell Data)	46
9.2. Animación con Archivo PVD	46
9.3. Salida en Texto Plano (Estilo FEAP)	47
9.4. Workflow ParaView	47
10. Ejemplos Completos	48
10.1. Marco 2D — Viga en Voladizo	48
10.2. Placa Continua con Plasticidad J2 (Gmsh)	49
10.3. Cable Pretensado en Régimen Corotacional	50

11. Uso Avanzado: API Programática	52
11.1. Patrón Fachada	52
11.2. Errores tipados	53
12. Diagnóstico de Problemas	54

Sobre este manual

Este manual está dirigido al **usuario final** de Solidum FEM: cubre la sintaxis del archivo de entrada `.yaml`, el catálogo completo de elementos, materiales y solvers, ejemplos de uso y el workflow de post-procesamiento. No entra al detalle de la implementación interna ni a la formulación matemática completa de cada componente.

Para la *referencia formal* de cada componente (ecuaciones, matrices **B**, criterios de aceptación), consultar `manuals/Reference_manual.pdf`, generado automáticamente desde `docs/specs/`.

Para la *visión arquitectural* del programa (capas, bloques funcionales, mecanismos transversales y dirección de evolución), consultar `manuals/Architecture_manual.pdf`.

Este manual se regenera con:

```
python manuals/build_user_manual.py
```

No editar este PDF manualmente; cualquier corrección debe hacerse sobre los archivos fuente en `manuals/sources/user/`.

Capítulo 1

Introducción y Configuración del Entorno

1.1 Filosofía del Proyecto

Solidum FEM nace como herramienta de investigación en mecánica del medio continuo y análisis no lineal de estructuras. Su diseño persigue dos objetivos simultáneos:

1. **Rigor numérico y físico.** Cada formulación incorporada está validada contra solución analítica o benchmark establecido. La separación estricta `Elemento` $\leftrightarrow$ `Material` permite combinar cualquier cinemática con cualquier ley constitutiva siempre que sus protocolos sean compatibles.
2. **Crecimiento sin fricción.** La arquitectura está optimizada para que añadir un elemento, material o solver nuevo sea barato. Auto-registro vía decoradores, contratos declarativos (`STRAIN_DIM`, `DOF_NAMES`) y validación temprana del input son los mecanismos centrales.

1.1.1 Modelo Data-Driven

Toda la orquestación de un análisis se concentra en un archivo `.yaml` legible por humanos. El usuario no necesita escribir código Python para definir un modelo; la API programática queda reservada para casos avanzados (optimización paramétrica, mallas generadas proceduralmente, acoplamiento con otros sistemas).

1.2 Requisitos del Sistema

Solidum FEM se ejecuta sobre Python 3.10 o superior en Windows, Linux y macOS sin compilación nativa.

1.2.1 Dependencias

```
pip install -e .           # desde la carpeta del proyecto: instala Solidum y sus
                             dependencias
```

- **numpy / scipy**: álgebra lineal densa y dispersa; SuperLU como solver directo disponible siempre.
- **pyyaml**: análisis sintáctico del archivo de entrada.
- **meshio**: importación de mallas Gmsh y exportación a VTK.
- **numba**: compilación JIT de los kernels críticos en elementos sólidos 2D y 3D y en los modelos de plasticidad.

1.2.2 Dependencias opcionales

Solidum funciona completo sin ellas; con ellas, los modelos grandes se resuelven mucho más rápido o caben en memoria. Se instalan como *extras*:

```
pip install -e .[fast]           # Intel MKL Pardiso: solver directo multihilo (
                                # recomendado)
pip install -e .[iterative]     # pyamg: preconditionador del solver iterativo
```

- **pypardiso** (`[fast]`): con él, el solver del sistema lineal —que en un análisis grande se lleva la mayor parte del tiempo— pasa a ser multihilo; entre 5 y 56 veces más rápido según el tamaño del modelo. Se usa automáticamente, sin tocar el YAML.
- **pyamg** (`[iterative]`): sólo lo usa el solver iterativo, que se pide expresamente (`linear_algebra: iterative`) para modelos demasiado grandes para la memoria. En Python 3.14 sobre Windows no trae binarios y hay que compilarlo; ver el anexo de la capa algebraica del manual de referencia.
- **scikit-sparse** (`[cholmod]`): factorización de Cholesky; en Windows se instala con **conda**, no con **pip**.

1.3 Herramientas Externas Recomendadas

- **Gmsh**: generador de mallas; produce archivos `.msh` consumibles directamente desde el YAML.
- **ParaView**: visualización interactiva de los archivos `.vtu` generados.

1.4 Ejecución de un Análisis

El runner principal vive en `examples/ejecutar_yaml.py`. Para ejecutar un modelo:

```
python examples/ejecutar_yaml.py ruta/al/modelo.yaml
```

El script carga el YAML, valida su contenido (acumulando *todos* los errores en una sola pasada antes de abortar), construye el dominio, ensambla el solver pedido, ejecuta el análisis y exporta los resultados según el bloque `output`.

Capítulo 2

Arquitectura del Programa

2.1 Separación Elemento $\leftrightarrow$ Material

Cada elemento finito declara dos contratos sobre el material que acepta:

- **STRAIN_DIM**: dimensión Voigt de la deformación que entrega al material. 1 para elementos axiales (truss, cable, marcos — la fibra centroidal); 3 para 2D (ϵ_{xx} , ϵ_{yy} , γ_{xy}); 6 para 3D continuo (ϵ_{xx} , ϵ_{yy} , ϵ_{zz} , γ_{xy} , γ_{yz} , γ_{xz}), [ADR 0012](#)).
- **DOF_NAMES**: lista de grados de libertad por nodo. Por ejemplo, ['ux', 'uy', 'rz'] para un marco 2D.

El protocolo del material es uniforme:

```
sigma, E_tangent, new_state = material.compute_state(epsilon, state_vars)
```

donde **epsilon** tiene dimensión **STRAIN_DIM**, **sigma** es el esfuerzo en la misma representación, **E_tangent** es la matriz tangente consistente y **state_vars** encapsula las variables internas (deformación plástica, daño acumulado, etc.).

Un material declara igualmente su **STRAIN_DIM**; el dominio rechaza al construir cualquier emparejamiento incompatible.

2.2 Auto-Registro de Componentes

Elementos, materiales y solvers se registran automáticamente al arrancar el programa mediante decoradores:

```
@ElementRegistry.register
class Truss2D(Element):
    DOF_NAMES = ['ux', 'uy']
    STRAIN_DIM = 1
    ...
```

Esto permite que el archivo YAML los referencie por nombre (**type: Truss2D**) sin que el usuario tenga que importar nada manualmente. El módulo **solidum.autodiscover** recorre los paquetes **elements/**, **materials/** y **math/solvers/** en la primera importación y puebla los registries.

2.3 Validación Temprana del Input

El parser YAML acumula todos los errores detectados en una sola pasada (no aborta al primero) y los presenta juntos. Comprueba:

- Ausencia de bloques obligatorios (`nodes`, `materials`, `elements`, `solver`).
- IDs duplicados de nodo, material o elemento.
- Referencias a materiales o nodos inexistentes desde `elements` y `boundary_conditions`.
- Tipos de elemento, material y solver no registrados.
- **Parámetros no aceptados por el constructor del elemento.** Si un elemento define `A` e `I` como parámetros, escribir `large_strains: true` en su entrada YAML produce un error explícito que indica los parámetros válidos para ese tipo.

Capítulo 3

Archivo de Entrada YAML

La interacción exclusiva con el motor de Solidum FEM se realiza mediante el archivo de entrada `.yaml`. Un archivo válido contiene los bloques siguientes (algunos opcionales según el caso de uso):

- `nodes` o `mesh` — geometría.
- `materials` — catálogo de leyes constitutivas instanciadas.
- `elements` — conectividad (omisible si se importa malla Gmsh).
- `boundary_conditions_by_node` / `_by_coord` / `_by_group` — restricciones de Dirichlet.
- `point_loads_by_node` / `_by_coord` / `_by_group` — cargas de Neumann puntuales.
- `solver` — tipo de solver y sus parámetros.
- `output` — frecuencia y formato de exportación de resultados.

3.1 Geometría: nodes y elements

Para modelos pequeños o estructuras reticulares (armaduras, marcos, cables), la geometría se escribe directamente en el YAML:

```
nodes:
- {id: 1, coords: [0.0, 0.0]}
- {id: 2, coords: [2.0, 0.0]}
- {id: 3, coords: [4.0, 0.0]}

elements:
- id: 1
  type: Frame2DEuler
  material: 1
  nodes: [1, 2]
  A: 0.01
  I: 8.33e-6
```

Las coordenadas pueden ser de 2 o 3 componentes; los elementos 1D 3D (`Truss3D`, `Cable3DCorot`, `Frame3D`) requieren las tres.

3.2 Importación de Malla Gmsh: mesh

Para problemas continuos 2D, se sustituye el bloque `nodes/elements` por una referencia al archivo `.msh`:

```
mesh: "geometria_placa.msh"
mesh_material: 1
mesh_thickness: 0.1
mesh_quadrature: "2x2"
```

Parámetro	Descripción
<code>mesh</code>	Ruta al archivo <code>.msh</code> de Gmsh, relativa al YAML.
<code>mesh_material</code>	ID del material por defecto que se asocia a toda la malla.
<code>mesh_thickness</code>	Espesor por defecto (estado plano 2D).
<code>mesh_quadrature</code>	Regla de Gauss: "2x2" (completa, recomendada) o "1x1" (reducida; produce <i>hourglassing</i>). La masa consistente y la capacidad térmica se integran siempre con la regla completa, con independencia de la de K.

3.2.1 Mapeo Avanzado: mesh_physical_groups

Si la geometría tiene zonas con materiales o espesores distintos definidos como *Physical Groups* en Gmsh, se mapean explícitamente:

```
mesh_physical_groups:
  "Zona_Hormigon":
    material: 1
    thickness: 0.20
    quadrature: "2x2"
  "Zona_Acero":
    material: 2
    thickness: 0.05
```

3.3 Materiales

Cada material se declara con un `id` único y un `type` registrado en el catálogo:

```
materials:
- id: 1
  type: Elastic1D
  E: 200.0e9
  density: 7850.0
- id: 2
  type: VonMises2D
  E: 210.0e9
  nu: 0.3
  sigma_y: 250.0e6
  H: 2.0e9
  hypothesis: plane_strain
  density: 7850.0
```

Los parámetros aceptados dependen del material; consulte el capítulo *Catálogo de Modelos Constitutivos* para el catálogo completo.

3.4 Condiciones de Frontera

Solidum FEM ofrece tres mecanismos para imponer restricciones cinemáticas (Dirichlet), elegibles según la facilidad operativa:

3.4.1 Por nodo: `boundary_conditions_by_node`

```
boundary_conditions_by_node:
- {node_id: 1, ux: 0.0, uy: 0.0, rz: 0.0}
- {node_id: 4, ux: 0.0, uy: 0.0}
```

3.4.2 Por coordenada: `boundary_conditions_by_coord`

Escanea todos los nodos y aplica la restricción a aquellos cuya coordenada coincide (dentro de `tol`):

```
boundary_conditions_by_coord:
x_min: {ux: 0.0, uy: 0.0, tol: 1.0e-5}      # empotrar lado izquierdo
rodillos: {coord: 'y', val: 10.0, uy: 0.0, tol: 1.0e-4}
```

Las claves predefinidas `x_min`, `x_max`, `y_min`, `y_max`, `z_min`, `z_max` usan los extremos detectados automáticamente del dominio; `coord` admite 'x', 'y' o 'z'. Los selectores funcionan igual con nodos 2D y 3D. Un selector que no casa con **ningún** nodo es un error de modelo y detiene la lectura con `ValueError` (no se ignora en silencio).

3.4.3 Por grupo físico: `boundary_conditions_by_group`

Aplica restricciones a todos los nodos de un *Physical Group* de Gmsh:

```
boundary_conditions_by_group:
"Borde_Empotrado": {ux: 0.0, uy: 0.0}
```

3.4.4 Restricciones multipunto (MPC): `linear_constraints`

Para apoyos en plano oblicuo, periodicidad de celda unitaria, uniones rígidas master-slave y simetrías no alineadas con los ejes globales, Solidum FEM admite restricciones afines lineales de la forma $u_s = g_s + \sum_i \alpha_{si} u_{m_i}$ (ADR 0004 fase 2). Se declaran en el bloque `linear_constraints`:

```
linear_constraints:
# Apoyo a 45 grados en el nodo 4: u_y = u_x.
- slave: {node: 4, dof: uy}
  masters:
  - {node: 4, dof: ux}
  coefficients: [1.0]

# Periodicidad: u_x del nodo 7 igual al del nodo 3.
- slave: {node: 7, dof: ux}
```

```

masters:
  - {node: 3, dof: ux}
coefficients: [1.0]

# Union rigida: u_x del satellite N9 sigue al maestro N5 con offset r_y=0.5.
- slave: {node: 9, dof: ux}
  masters:
    - {node: 5, dof: ux}
    - {node: 5, dof: rz}
  coefficients: [1.0, -0.5]
g: 0.0

```

Las restricciones se imponen por eliminación directa (no por penalización), de modo que la imposición es exacta a redondeo y preserva la simetría y la positividad definida del sistema. Las cadenas master-slave se resuelven automáticamente por cierre transitivo; los ciclos y las redeclaraciones inconsistentes se detectan en construcción y abortan el análisis con un mensaje específico.

3.5 Cargas Puntuales

Misma estructura que las condiciones de frontera, en bloques `point_loads_by_node`, `point_loads_by_coord` y `point_loads_by_group`. Los valores son fuerzas nodales (en las unidades del modelo, típicamente Newtons):

```

point_loads_by_node:
  - {node_id: 3, uy: -15000.0}

point_loads_by_group:
  "Borde_Carga": {ux: 1500000.0}

```

Las cargas son **estrictas**: un `node_id` inexistente se rechaza en la validación del archivo, y un nombre de DOF que el nodo no tiene (`fy` en vez de `uy`, `rz` sobre una armadura) o un grupo físico inexistente detienen la lectura con `ValueError`. Antes se ignoraban en silencio y el análisis corría sin carga.

En los **solvers transitorios mecánicos** (`NewmarkSolver`, `HHTSolver`, `NewtonNewmarkSolver`, `NewtonHHTSolver`, `CentralDifferenceSolver`) las cargas del archivo —puntuales más peso propio o fuerza de cuerpo— se aplican como **escalón constante en el tiempo** desde $t = 0$ (`F_func = cte`). Una historia $\mathbf{F}(t)$ arbitraria requiere construir el solver desde Python con su propio `F_func` (§10).

3.6 Fuerzas de Cuerpo

Una *fuerza de cuerpo* es una fuerza por unidad de volumen $\mathbf{b}$ que actúa sobre todo el dominio material del elemento, no solo sobre su frontera. Cada elemento del catálogo (armaduras, cables, marcos 2D/3D, sólidos 2D) integra la contribución consistente $\int \mathbf{N}^T \mathbf{b} A dx$ y la acumula al vector global de fuerzas. Para vigas Hermite cúbicas el reparto incluye fuerza nodal $qL/2$ y momento $\pm qL^2/12$; para barras axiales es mitad-mitad por nodo.

Casos típicos en mecánica estructural: peso propio (un caso particular con $\mathbf{b} = \rho \mathbf{g}$), fuerzas centrífugas en cuerpos rotatorios ($\mathbf{b} = \rho \omega^2 \mathbf{r}$), fuerzas electromagnéticas sobre materiales conductores,

expansión térmica modelada como pseudocarga. Solidum expone dos formas declarativas en YAML que cubren todos estos casos.

3.6.1 Forma general: `body_force`

Aplica un vector $\mathbf{b}$ uniforme (fuerza por unidad de volumen, en ejes globales) a todos los elementos del modelo:

```
body_force: [0.0, -77008.5]          # 2D: bx, by en N/m^3
```

o, para problemas 3D:

```
body_force: [0.0, 0.0, -77008.5]    # 3D: bx, by, bz
```

Es la forma genérica y la única necesaria para fuerzas de cuerpo que no son peso propio (campos electromagnéticos precalculados, centrífuga estática, etc.). También sirve para peso propio cuando todo el modelo es monomaterial y el usuario prefiere precalcular $\rho \cdot g$.

3.6.2 Caso particular — peso propio: `gravity` + `density`

El peso propio es el caso especial $\mathbf{b} = \rho \mathbf{g}$. Para problemas multimaterial — donde cada elemento debe ver *su propio* ρ — Solidum expone un atajo declarativo: la densidad como propiedad del material, y un vector global de gravedad.

```
materials:
- {id: 1, type: Elastic1D, E: 210.0e9, density: 7850.0}    # acero
- {id: 2, type: Elastic1D, E: 30.0e9, density: 2500.0}     # hormigón

gravity: [0.0, -9.81]                                     # m/s^2, +y hacia arriba
```

Qué hace Solidum. Para cada elemento computa $\mathbf{b}_e = \rho_e \cdot \mathbf{g}$, donde ρ_e es la densidad del material asignado, e integra la contribución consistente con la misma maquinaria de `body_force`. Resultado: estructuras multimaterial dan peso propio físicamente correcto, cada elemento con su propio ρ .

Densidad solo cuando se usa. El campo `density` es **opcional** al declarar un material ([ADR 0008](#)): los análisis estáticos puros que no invocan peso propio ni matriz de masa no necesitan declararla. Pero al invocar `gravity` (o, en su día, análisis modal/dinámico), Solidum exige la densidad de cada material involucrado: si algún material no la trae declarada, el cálculo falla con `ValueError` listando los materiales afectados por nombre. Imposible propagar masa cero silenciosa. Si un material legítimamente carece de masa física (penalty, restricción), declararlo explícitamente como `density: 0.0` — caso en el cual `gravity` emite un `WARNING` informativo y su contribución al peso propio es nula.

3.6.3 Detalles transversales

Exclusividad. `body_force` y `gravity` son dos formas declarativas para expresar fuerzas de cuerpo; declarar ambas en el mismo archivo lanza error de validación. La forma general (`body_force`) es la subyacente; `gravity` es atajo para su caso particular más común.

Padding 2D↔3D. Si el modelo mezcla elementos 2D y 3D, declarar el vector con tres componentes; el ensamblador pasa el slice apropiado a cada elemento según la dimensionalidad declarada por su `DOF_NAMES`.

Reservado para análisis dinámico. El atributo `density` introducido aquí es la misma magnitud que consume la matriz de masa $\mathbf{M}_e = \int \rho \mathbf{N}^\top \mathbf{N} dV$ en análisis modales y dinámicos. Declararlo prepara el modelo para esas extensiones sin coste adicional ([ADR 0008](#)).

3.7 Solver

```
solver:
  type: NonlinearSolver
  num_steps: 20
  max_iter: 15
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5
```

Los solvers disponibles y sus parámetros se documentan en los capítulos *Esquemas de Solución* y *Análisis Dinámico*.

El solver del sistema lineal que cada uno resuelve internamente **lo elige Solidum** según las propiedades de la matriz; no hace falta declararlo. El campo opcional `linear_algebra` sirve para diagnóstico o para pedir el solver iterativo en modelos muy grandes:

```
solver:
  type: LinearSolver
  linear_algebra: iterative      # auto (default), lu, pardiso, cholesky, iterative[:amg|
                                jACOBI|none]
```

Un valor desconocido, o `iterative:amg` sin `pyamg` instalado, se rechaza al leer el archivo.

3.7.1 Bloque convergence

Los solvers no lineales (`NonlinearSolver`, `ArcLengthSolver`) aceptan un bloque `convergence` que configura el criterio de parada de las iteraciones de Newton. La política sigue el patrón estándar de tolerancias mixtas absoluta + relativa, separado por dos criterios físicos: *fuerza* (residuo de equilibrio) y *desplazamiento* (incremento del iterado). La forma exacta de la comparación es:

```
||R[libres]|| <= atol_force + rtol_force * max(||F_ext||, ||F_int||)
||dU||        <= atol_disp  + rtol_disp  * ||U||
```

Ambos criterios deben cumplirse **simultáneamente** (semántica AND) para dar el paso por convergido.

Parámetros disponibles (todos opcionales, con defaults razonables):

- `rtol_force` (default 1.0e-5): tolerancia relativa del residuo de fuerza. Es el parámetro que normalmente ajustarás: bajarlo a 1.0e-7 para análisis de precisión, subirlo a 1.0e-3 para exploración rápida.
- `rtol_disp` (default 1.0e-5): tolerancia relativa de la corrección de desplazamiento. En problemas casi-rígidos puede afinarse independientemente del de fuerza.

- `atol_force_factor` (default $1.0\text{e-}9$): factor adimensional que define el piso absoluto de fuerza. La tolerancia absoluta efectiva se autoderiva como `atol_force_factor · escala_inicial`, donde la escala se calcula en el primer ensamblaje. Solo conviene ajustarlo si el análisis tiene tramos donde la carga característica colapsa transitoriamente y aparece falsa no-convergencia.
- `atol_disp_factor` (default $1.0\text{e-}9$): análogo para desplazamiento.

Por qué los atol se autoderivan. El término absoluto vive en las unidades del problema (newtons para fuerza, metros para desplazamiento). Codificarlo como constante global obligaría a ajustarlo al cambiar de unidades (N/m, kN/mm, MPa/mm...). En su lugar, Solidum calcula la escala característica del problema en el primer ensamblaje y multiplica por el factor adimensional. El resultado: **el mismo análisis planteado en distintos sistemas de unidades converge en el mismo número de iteraciones hasta paridad de bits**, sin tocar las tolerancias. La justificación arquitectural está en el [ADR 0007](#).

Cuándo afinar. Para la inmensa mayoría de análisis los defaults son suficientes. Considera ajustar:

- `rtol_force` y `rtol_disp` más estrictos cuando publiques resultados o compares contra solución analítica ($1.0\text{e-}7$ a $1.0\text{e-}9$).
- `rtol_disp` más laxo que `rtol_force` en problemas casi-rígidos donde el incremento de desplazamiento es naturalmente pequeño y baja muy rápido, dejando que el criterio de fuerza domine.
- Los factores `atol_*` prácticamente nunca; solo si encuentras un análisis específico donde la escala colapsa y los logs muestran iteraciones oscilando cerca del umbral.

Nota sobre el solver algebraico interno. El sistema lineal $\mathbf{K} \delta \mathbf{U} = \mathbf{R}$ que aparece dentro de cada iteración se resuelve con un backend algebraico (Cholesky, LU...) seleccionado *automáticamente* según las propiedades de $\mathbf{K}$. Esta elección **no** es decisión de modelado y no requiere configuración. Si por motivos de diagnóstico necesitas forzar un backend específico, consulta el capítulo *Anexos técnicos — Capa algebraica* del Manual de Referencia.

3.8 Salida: output

Configura qué se exporta y cuándo:

```
output:
  file_name: "resultados_placa"
  pre_process: {export: true}      # exporta el modelo sin deformar (paso 0)
  results:
    frequency: "all_steps"        # "last_step" o "all_steps"
    nodal_results: ["Displacements"]
    element_results: ["Von_Mises", "Internal_State", "Sigma_XX"]
  text_export:                    # opcional: salida estilo FEAP en .txt
    export: true
    nodes: all
    elements: all
```

Nota

Cuando `frequency: "all_steps"` y el solver es no lineal, se genera un archivo `.vtu` por paso convergido más un archivo maestro `.pvd` que ParaView interpreta como animación.

Capítulo 4

Catálogo de Elementos Finitos

El motor expone varias familias de elementos: **1D** (estructuras reticulares: armaduras, cables, marcos), **2D** (continuo plano: cuadrilátero y triángulo), **3D** (continuo tridimensional: hexaedros y tetraedros, lineales y cuadráticos), **discontinuidades embebidas** (fractura con salto interior) y **térmicos** (conducción de calor, con un grado de libertad escalar por nodo; se documentan en el capítulo «Análisis Térmico»). Todos se referencian desde el YAML por su nombre exacto en `type:`.

4.1 Elementos 1D

4.1.1 Armaduras: Truss2D / Truss2DCorot / Truss3D / Truss3DCorot

Barra biarticulada que transmite exclusivamente fuerza axial. Cuatro variantes según dimensión y régimen geométrico:

Tabla 4.1: Familia de armaduras.

Elemento	DOFs/nodo	Dimensión	Régimen geom.	Uso
Truss2D	ux,uy	2D	lineal	cargas pequeñas, rotaciones pequeñas
Truss2DCorot	ux,uy	2D	corotacional	grandes desplazamientos y rotaciones planas
Truss3D	ux,uy,uz	3D	lineal	armaduras espaciales en régimen lineal
Truss3DCorot	ux,uy,uz	3D	corotacional	grandes rotaciones en el espacio

Parámetros: A (área de la sección transversal). Convención de signos: $\sigma > 0 \Leftrightarrow \varepsilon > 0$ (elongación).

Régimen de validez: $|\varepsilon| \lesssim 10^{-2}$. Las variantes **Corot** aceptan rotaciones de cualquier magnitud entre commits; las versiones lineales asumen rotaciones pequeñas.

```
elements:
- id: 1
  type: Truss2DCorot
  material: 1
```

```
nodes: [1, 2]
A: 0.0005
```

4.1.2 Cables: Cable2DCorot / Cable3DCorot

Elemento 1D que transmite *únicamente* fuerza axial de tensión (no resiste compresión). La unilateralidad la aporta el material (típicamente `CableMaterial1D`, ver capítulo *Catálogo de Modelos Constitutivos*); el elemento implementa la cinemática corotacional y la transferencia de la deformación al material.

- **DOFs/nodo:** u_x, u_y (2D); u_x, u_y, u_z (3D).
- **Parámetros:** A .
- **Cinemática:** corotacional (longitud y cosenos directores se recalculan en cada evaluación).
- **Régimen de validez:** $|\varepsilon| \lesssim 10^{-2}$ tensado. En estado destensado el elemento aporta rigidez nula al sistema global; otros elementos deben garantizar la estabilidad numérica.

Advertencia

Un cable completamente destensado ($\sigma = 0$) tiene $\mathbf{K}_T = \mathbf{0}$ en sus DOFs. Si todos los caminos de carga del modelo dependen del cable y este se destensa, la matriz tangente global se vuelve singular. Pretensar el cable o garantizar redundancia estructural.

```
materials:
- id: 1
  type: CableMaterial1D
  E: 150.0e9
  density: 7850.0

elements:
- id: 1
  type: Cable2DCorot
  material: 1
  nodes: [1, 2]
  A: 5.0e-5
```

4.1.3 Marcos 2D: Frame2DEuler / Frame2DTimoshenko / Frame2DEulerCorot

Elementos viga 2D que transmiten axial, cortante y momento flector. Tres variantes:

- **Frame2DEuler:** vigas esbeltas ($L/h \gtrsim 10$), Euler-Bernoulli, régimen geoméricamente lineal. Parámetros: A, I .
- **Frame2DTimoshenko:** vigas peraltadas o cortas ($L/h \lesssim 10$). Incluye deformación por cortante; corrige automáticamente *shear locking* en el límite esbelto. Parámetros: A, I, A_s (área efectiva de cortante), ν (opcional si el material expone ν).

- **Frame2DEulerCorot**: variante corotacional de Euler-Bernoulli. Captura grandes desplazamientos y grandes rotaciones rígidas del elemento (rotaciones deformacionales nodales moderadas, $|\bar{\theta}| \lesssim 30^\circ$). Parámetros: A , I .

DOFs/nodo: u_x , u_y , r_z . Convención: $\sigma > 0 \Leftrightarrow \varepsilon > 0$ (elongación) en el eje axial; r_z positivo antihorario.

Advertencia

Plasticidad por flexión. Los elementos marco pasan al material únicamente la deformación axial centroidal $(u_j - u_i)/L$; el módulo tangente E_t devuelto escala toda la matriz local (axial y flexión por igual). Esto significa que los marcos reproducen correctamente *plasticidad axial pura* pero *no* forman rótulas plásticas por flexión: la fluencia por momento requiere integración de $\sigma(y)$ sobre la sección, característica que se incorporará en una clase **FiberSection** futura.

```
elements:
- id: 1
  type: Frame2DEulerCorot
  material: 1
  nodes: [1, 2]
  A: 0.01
  I: 8.33e-6

- id: 2
  type: Frame2DTimoshenko
  material: 1
  nodes: [2, 3]
  A: 0.01
  I: 8.33e-6
  As: 0.00833
```

4.1.4 Marco 3D: Frame3D

Viga 3D Euler-Bernoulli con 6 DOFs/nodo (u_x , u_y , u_z , r_x , r_y , r_z). Transmite axial, cortantes en dos planos, flexiones en dos planos y torsión de Saint-Venant pura. Régimen geométricamente lineal (la variante corotacional 3D queda como pendiente).

Parámetros:

- A — área de la sección.
- I_y , I_z — momentos de inercia respecto a los ejes locales y , z .
- J — constante torsional de Saint-Venant.
- ν — coeficiente de Poisson (opcional; se toma del material si lo expone).
- **ref_vector** — vector de referencia (opcional, default $[0,0,1]$) que fija la orientación de los ejes locales y , z de la sección. Para barras casi-verticales se sugiere fijarlo explícitamente.

```
elements:
- id: 1
  type: Frame3D
  material: 1
  nodes: [1, 2]
  A: 0.01
  Iy: 8.33e-6
  Iz: 8.33e-6
  J: 1.66e-5
  ref_vector: [0.0, 0.0, 1.0]
```

4.2 Elementos 2D

La importación de mallas desde Gmsh instancia automáticamente los elementos 2D según la topología detectada (cuadriláteros $\rightarrow$ Quad4; triángulos $\rightarrow$ Tri3). También pueden declararse manualmente desde el bloque `elements`.

4.2.1 Cuadrilátero Bilineal: Quad4

Elemento isoparamétrico de 4 nodos, integración Gauss configurable.

- **DOFs/nodo:** u_x , u_y ($\text{STRAIN_DIM} = 3$, Voigt $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$).
- **Nodos:** 4, ordenados en sentido antihorario.
- **Cuadratura:** "2x2" por defecto (4 puntos). "1x1" disponible pero produce modos espurios (*hourglassing*).
- **Parámetros:** `thickness`, `quadrature` (opcional).
- **Implementación:** kernels críticos compilados con `@njit` (Numba).
- **Limitación:** bloqueo volumétrico con materiales casi-incompresibles ($\nu \rightarrow 0.5$); en ese régimen requeriría formulación mixta (no implementada).

4.2.2 Triángulo de Deformación Constante: Tri3

Triángulo CST de 3 nodos, un único punto de integración.

- **DOFs/nodo:** u_x , u_y ($\text{STRAIN_DIM} = 3$).
- **Cuadratura:** 1 punto (deformación uniforme).
- **Parámetros:** `thickness`.
- **Limitación:** *shear locking* severo, convergencia lenta. **Preferir** Quad4 salvo en transiciones donde Quad4 no encaja geométricamente.

4.2.3 Cuadrilátero Serendípito de Orden 2: Quad8

Cuadrilátero isoparamétrico cuadrático de 8 nodos (4 vértices antihorarios + 4 nodos medios de borde). Funciones de forma serendípitas (sin el término $\xi^2\eta^2$). Reproduce campos cuadráticos exactamente y mitiga el bloqueo por cortante en problemas de flexión.

- **DOFs/nodo:** `ux`, `uy` (`STRAIN_DIM` = 3).
- **Cuadratura:** Gauss 3×3 (9 puntos) por defecto; 2×2 disponible vía `quadrature` pero con riesgo de modos espurios.
- **Parámetros:** `thickness`, `quadrature` (opcional).
- **Tracción de borde:** reparte 1/6, 4/6, 1/6 entre los nodos del borde (vértice, medio, vértice).

4.2.4 Cuadrilátero Lagrangiano de Orden 2: Quad9

Cuadrilátero Lagrangiano cuadrático de 9 nodos (los 8 de **Quad8** más un noveno nodo central interior). Las funciones de forma son producto tensorial Lagrange 1D-1D, espacio polinómico completo Q_2 .

- **DOFs/nodo:** `ux`, `uy` (`STRAIN_DIM` = 3).
- **Cuadratura:** Gauss 3×3 .
- **Parámetros:** `thickness`, `quadrature` (opcional).
- **Diferencia con Quad8:** el nodo central interior añade el término $\xi^2\eta^2$ y mejora el comportamiento en problemas con campos no separables.

4.2.5 Triángulo Cuadrático Completo P_2 : Tri6

Triángulo isoparamétrico cuadrático de 6 nodos (3 vértices + 3 medios de borde). Resuelve el *shear locking* severo de **Tri3** y reproduce campos cuadráticos exactamente.

- **DOFs/nodo:** `ux`, `uy` (`STRAIN_DIM` = 3).
- **Cuadratura:** 3 puntos en los puntos medios (regla `tri_3`).
- **Parámetros:** `thickness`.
- **Tracción de borde:** reparte 1/6, 4/6, 1/6 entre los nodos del borde (vértice, medio, vértice).
- **Cuándo usarlo:** transiciones de malla cuadráticas, geometrías curvadas donde **Quad8/Quad9** no encajan.

```
elements:
- {id: 1, type: Quad8, material: 1, thickness: 0.1, nodes: [1, 2, 3, 4, 5, 6, 7, 8]}
- {id: 2, type: Quad9, material: 1, thickness: 0.1, nodes: [1, 2, 3, 4, 5, 6, 7, 8,
  9]}
- {id: 3, type: Tri6, material: 1, thickness: 0.1, nodes: [1, 2, 3, 4, 5, 6]}
```

4.3 Elementos 2D con discontinuidad embebida

Subfamilia dedicada a **fractura computacional**: el elemento materializa una discontinuidad interna Γ_d cuando se cumple un criterio de activación, y enriquece su cinemática con un salto de desplazamientos $\llbracket u \rrbracket$ gobernado por un material cohesivo. Los grados de libertad del salto son **elementales**: se condensan dentro del elemento y nunca llegan al ensamblador, de modo que el tamaño del sistema global no cambia.

4.3.1 Triángulo CST con Discontinuidad Interior: CST_Embedded2D

CST de 3 nodos con cinemática KOS enriquecida (Retama 2010, Caps. 2, 5, 6 y 7).

- **DOFs/nodo**: u_x , u_y ($\text{STRAIN_DIM} = 3$). Los 2 DOFs del salto son elementales, no globales.
- **Cuadratura**: 1 punto (hereda del Tri3).
- **Parámetros**: `thickness`, `material` (bulk), `cohesive_material`, `activation_criterion` (opcional, default `rankine`).
- **Dos materiales**: a diferencia del resto del catálogo, requiere **un material de bulk** que gobierna el continuo y **uno cohesivo** que gobierna el salto en Γ_d . En YAML son dos campos distintos.
- **Estado intacto**: idéntico al Tri3 hasta que la discontinuidad se activa.
- **Activación**: criterio de Rankine ($\sigma_I > \sigma_{t0}$ del cohesivo) evaluado en el centroide con el estado convergido del paso anterior. Es **irreversible**: una vez activada persiste aunque el paso siguiente descargue.
- **Bulk aceptado**: sólo Elastic2D en la fase actual — la *discrete approach* presupone bulk elástico (ADR 0010).
- **Post-proceso**: `compute_gauss_state(U)` añade la clave '`discontinuity`' con normal, tangente, centroide, l_d , salto, tracción y daño cuando el elemento está agrietado.
- **Limitación operativa**: el trazado completo de la rama post-pico requiere un solver capaz de atravesar el softening con penalty cohesivo rígido. Ver el capítulo de diagnóstico.

```
cohesive_materials:
- {id: 1, type: CohesiveDamageIsotropic, sigma_t0: 2.5e6, G_f: 100.0,
  K_e: 1.0e13, softening: linear}
elements:
- {id: 1, type: CST_Embedded2D, nodes: [1, 2, 3],
  material: 1, cohesive_material: 1}
```


4.4 Elementos 3D

Sólidos tridimensionales isoparamétricos ([ADR 0012](#)). Todos comparten DOFs `ux`, `uy`, `uz` y `STRAIN_DIM = 6` sobre la convención Voigt 3D del proyecto, $[\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]$, y exigen un material 3D (`Elastic3D`, `VonMises3D`, `DruckerPrager3D`, `IsotropicDamage3D`).

A diferencia de los 2D, **no llevan thickness**: el volumen sale de la geometría. Las cargas de superficie se aplican por **cara numerada con normal saliente** mediante `compute_face_traction(face,  $\bar{\mathbf{t}}$ )`, con $\bar{\mathbf{t}}$ expresada en ejes globales.

4.4.1 Hexaedro Trilineal: Hex8

Hexaedro isoparamétrico de 8 nodos, espejo natural del `Quad4`. Orden de nodos `VTX_HEXA-HEDRON`.

- **Nodos:** 8 · **Cuadratura:** Gauss $2 \times 2 \times 2$ (8 puntos) por defecto.
- **Parámetros:** `quadrature` (opcional; `hex_3x3x3` para no lineales severos, `hex_1x1x1` reducida con riesgo de *hourglass*).
- **Caras:** 6, numeradas 0 ($-\zeta$), 1 ($+\zeta$), 2 ($-\eta$), 3 ($+\xi$), 4 ($+\eta$), 5 ($-\xi$).
- **Limitaciones:** bloqueo volumétrico con $\nu \rightarrow 0.5$ y *shear locking* con una sola capa de elementos en la sección. Sin mitigación, por política idéntica a la del `Quad4`.

4.4.2 Tetraedro Lineal: Tet4

Tetraedro de 4 nodos, CST 3D — espejo del `Tri3`. Deformación uniforme en el elemento.

- **Nodos:** 4 · **Cuadratura:** 1 punto baricéntrico.
- **Caras:** 4 triangulares; la cara i es la opuesta al nodo i .
- **Limitación:** *shear locking* severo, peor que el `Hex8` por la pobreza del espacio lineal, y bloqueo volumétrico aún más acusado. **Preferir** `Hex8` en mallas hexaédricas; reservar `Tet4` para transiciones o geometrías que no admitan hexaedros.

4.4.3 Hexaedro Serendípito de Orden 2: Hex20

Hexaedro cuadrático de 20 nodos (8 vértices + 12 medios de arista, sin nodos de cara ni centroide). Análogo 3D del `Quad8`.

- **Nodos:** 20 · **Cuadratura:** Gauss $3 \times 3 \times 3$ (27 puntos) por defecto.
- **Caras:** 6, de 8 nodos cada una (cara `Quad8`).
- **Tracción de cara:** reparto serendípito — cada vértice recibe $-A\bar{t}/12$ y cada medio $4A\bar{t}/12$. Los signos negativos en los vértices son el fenómeno serendípito conocido del `Quad8` (Cook-Malkus-Plesha-Witt §6.5), no un error.

- **Ventaja medida:** en la viga esbelta de MacNeal-Harder, una malla $6 \times 1 \times 1$ de **Hex20** alcanza el 97 % de la deflexión de Euler-Bernoulli, frente a menos del 55 % de un **Hex8** $12 \times 1 \times 1$.
- **Precaución:** con **hex_2x2x2** aparecen 6 modos de *hourglass* por elemento aislado, sin estabilización. Usar la cuadratura por defecto.

4.4.4 Hexaedro Lagrangiano Triquadrático: Hex27

Hexaedro Lagrangiano completo de 27 nodos (20 del **Hex20** + 6 centros de cara + 1 centro de cuerpo). Análogo 3D del **Quad9**.

- **Nodos:** 27 · **Cuadratura:** Gauss $3 \times 3 \times 3$.
- **Caras:** 6, de 9 nodos cada una (cara **Quad9**).
- **Diferencia con Hex20:** reproduce **todos** los polinomios triquadráticos, incluidos los términos $\xi^2 \eta^2 \zeta^2$ que faltan en el serendípito. En flexión simple con geometría rectilínea ambos dan resultados prácticamente idénticos, con mayor coste para el **Hex27**; la diferencia se paga en geometrías de curvatura severa (Bathe FEP §5.3.2).
- **Precaución:** con **hex_2x2x2** aparecen 27 modos de *hourglass* por elemento aislado — la combinación más problemática del catálogo 3D. Usar $3 \times 3 \times 3$ siempre.

4.4.5 Tetraedro Cuadrático: Tet10

Tetraedro de 10 nodos (4 vértices + 6 medios de arista). Análogo 3D del **Tri6**.

- **Nodos:** 10 · **Cuadratura:** Stroud **tet_4** (4 puntos) por defecto; **tet_15** (Keast, 15 puntos) para elementos distorsionados.
- **Caras:** 4 triangulares de 6 nodos (cara **Tri6**).
- **Masa:** la matriz consistente usa **tet_15** fijo, independientemente de la cuadratura elegida para K , para integrar exactamente el producto cuadrático×cuadrático en análisis modal y transitorio.
- **Cuándo usarlo:** mallas no estructuradas y geometrías complejas donde un mapeo hexaédrico no es viable. Mitiga drásticamente ambos bloqueos del **Tet4**.
- **Limitación conocida:** sobre **superficies curvas** malladas por descomposición de hexaedros en tetraedros, la representación isoparamétrica se degrada porque parte de los nodos medios quedan sobre aristas rectas. Sobre geometría plana el elemento es exacto. Para geometría curva con tetraedros conviene un mallador nativo que sitúe los nodos medios sobre la superficie.

```
elements:
- {id: 1, type: Hex8, material: 1, nodes: [1, 2, 3, 4, 5, 6, 7, 8]}
- {id: 2, type: Tet4, material: 1, nodes: [1, 2, 3, 4]}
- {id: 3, type: Hex20, material: 1, nodes: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
                                           11, 12, 13, 14, 15, 16, 17, 18, 19, 20]}
- {id: 4, type: Tet10, material: 1, nodes: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]}
```

Capítulo 5

Catálogo de Modelos Constitutivos

5.1 Elasticidad Lineal

5.1.1 Elastic1D — elasticidad lineal axial

Ley de Hooke escalar $\sigma = E \cdot \varepsilon$. Sin variables internas. Compatible con todos los elementos 1D (armaduras y marcos).

```
materials:
- id: 1
  type: Elastic1D
  E: 200.0e9
  density: 7850.0
```

5.1.2 Elastic2D — elasticidad lineal isotrópica 2D

Tensor constitutivo isotrópico $\sigma = \mathbf{C} \cdot \varepsilon$ en notación Voigt $[xx, yy, xy]$. Sin variables internas. Compatible con Quad4, Tri3.

```
materials:
- id: 1
  type: Elastic2D
  E: 200.0e9
  nu: 0.3
  hypothesis: plane_stress
  density: 7850.0
```

La hipótesis `plane_stress` se usa para placas delgadas sin confinamiento normal; `plane_strain` para secciones de presa, túneles o problemas con extensión infinita en el eje longitudinal.

5.2 Plasticidad

5.2.1 Elastoplastic1D — plasticidad J2 axial con endurecimiento isotrópico

Plasticidad asociativa con criterio $f = |\sigma_{\text{trial}}| - (\sigma_y + H\alpha) \leq 0$. Algoritmo de *return mapping* clásico 1D; tangente algorítmica consistente $E_t = E H / (E + H)$.

- **Parámetros:** `E`, `sigma_y` (fluencia inicial), `H` (módulo de endurecimiento; `H=0` = perfectamente plástico).
- **Variables internas:** ε_p (deformación plástica), α (acumulada equivalente).
- **Compatible con:** armaduras y marcos 1D (en marcos solo se aplica al esfuerzo axial; ver advertencia en el capítulo *Catálogo de Elementos Finitos*).

```
materials:
- id: 1
  type: Elastoplastic1D
  E: 200.0e9
  sigma_y: 250.0e6
  H: 2.0e9
  density: 7850.0
```

5.2.2 VonMises2D — plasticidad J2 plana con endurecimiento isótropo

Plasticidad asociativa con criterio J2 y endurecimiento isótropo lineal. Soporta **dos hipótesis cinemáticas** mutuamente excluyentes, seleccionadas con el campo `hypothesis` al construir el material:

- **plane_strain** ($\varepsilon_{zz} = 0$): return mapping radial cerrado sobre la parte desviadora 3D extendida (Simó-Hughes §3.3). Corrector $\Delta\gamma = f_{\text{trial}}/(2G + \frac{2}{3}H)$ con $N = s_{\text{trial}}/\|s_{\text{trial}}\|$ y actualización $\alpha_{\text{new}} = \alpha + \sqrt{2/3}\Delta\gamma$. Default histórico.
- **plane_stress** ($\sigma_{zz} = 0$): *plane stress projected algorithm* (Simó-Hughes §3.4.1). Función de fluencia proyectada $\bar{f} = \frac{1}{2}\boldsymbol{\sigma}^T \mathbf{P} \boldsymbol{\sigma} - R^2/3$ y Newton local escalar sobre $\Delta\gamma$. Converge en 3–6 iteraciones por evaluación. Actualización físicamente correcta $\alpha_{\text{new}} = \alpha + \Delta\gamma\sqrt{2\boldsymbol{\sigma}^T \mathbf{P} \boldsymbol{\sigma}}/3$ (en plane stress $\Delta\gamma$ tiene unidades [1/esfuerzo], por lo que la heurística $\sqrt{2/3}\Delta\gamma$ válida en plane strain rompe la invariancia bajo cambio de unidades). Incompresibilidad plástica cierra $e_{zz}^p = -(e_{xx}^p + e_{yy}^p)$.

Parámetros comunes: `E`, `nu`, `sigma_y`, `H` (≥ 0 , default 0 = perfectamente plástico), `hypothesis` (default `plane_strain`), `density` (opcional). Variables internas: ε_p tensorial 4 componentes $[xx, yy, zz, xy_{\text{tens}}]$ y α acumulada equivalente adimensional. **Tangente algorítmica consistente cerrada** en ambas hipótesis (con corrección por $d\alpha/d\Delta\gamma \neq \sqrt{2/3}$ en plane stress). Compatible con todos los elementos 2D del catálogo.

```
materials:
# Hipótesis por defecto: plane_strain
- id: 2
  type: VonMises2D
  E: 210.0e9
  nu: 0.3
  sigma_y: 250.0e6
  H: 2.0e9
  hypothesis: plane_strain
  density: 7850.0
```

```
# Mismo material en plane_stress (placas delgadas sin confinamiento normal)
- id: 3
  type: VonMises2D
  E: 210.0e9
  nu: 0.3
  sigma_y: 250.0e6
  H: 2.0e9
  hypothesis: plane_stress
  density: 7850.0
```

5.2.3 DruckerPrager2D — plasticidad friccional cohesivo-friccional 2D

Modelo de Drucker-Prager: cono circular suave de Mohr-Coulomb con cohesión y fricción interna. Criterio de fluencia $f = \sqrt{J_2} + \eta_f I_1 - k(\alpha) \leq 0$ con $k(\alpha) = k_0 + H\alpha$ (endurecimiento isótropo lineal en cohesión). Plasticidad *no* asociada por defecto: el ángulo de dilatación ψ es parámetro independiente del ángulo de fricción ϕ .

Algoritmo — dos ramas con detección automática:

- **Return regular** (superficie del cono): $\Delta\gamma = f_{\text{trial}}/(G + 9K\eta_f\eta_g + H)$, dirección desviadora preservada. Tangente algorítmica $\mathbf{C}_{\text{alg}} = K\mathbf{v} \otimes \mathbf{v} + 2G(1 - \beta)\mathbf{I}_{\text{dev}} + 4G\beta\hat{\mathbf{n}} \otimes \hat{\mathbf{n}} - (1/A)\mathbf{b}_g \otimes \mathbf{b}_f$ con $\beta = G\Delta\gamma/\sqrt{J_2^{\text{trial}}}$.
- **Return al ápice** (vértice puramente hidrostático): si tras el return regular el estado caería fuera del cono, conmuta a $\Delta\gamma_{\text{apex}} = (I_1^{\text{trial}}\eta_f - k(\alpha))/(9K\eta_f\eta_g + H)$ con $\sigma_{n+1} = (k/3\eta_f)\mathbf{I}$ puramente hidrostático. Tangente reducida $KH/(9K\eta_f\eta_g + H)\mathbf{v} \otimes \mathbf{v}$.

Calibración con Mohr-Coulomb (campo variant):

- **plane_strain_matched** (default): coincide exactamente con MC en plane strain. $\eta_f = \tan\phi/\sqrt{9 + 12\tan^2\phi}$, $k_0 = 3c_0/\sqrt{9 + 12\tan^2\phi}$.
- **outer_cone**: circunscribe MC. $\eta_f = 2\sin\phi/[\sqrt{3}(3 - \sin\phi)]$.
- **inner_cone**: inscribe MC. $\eta_f = 2\sin\phi/[\sqrt{3}(3 + \sin\phi)]$.

Parámetros: E, nu, cohesion (cohesión inicial c_0), phi_deg ($0 \leq \phi < 90$), psi_deg (dilatación, $0 \leq \psi \leq \phi$; default $\psi = \phi$ = asociada), H (≥ 0 , default 0), variant (default **plane_strain_matched**), density (opcional). Variables internas: ε_p tensorial 4 componentes y α acumulada. **IS_SYMMETRIC = False** declarativo (no asociada $\Rightarrow$ tangente asimétrica; el despachador algebraico **ADR 0003** elige LU).

Advertencia

Restricción cinemática: solo **plane_strain**. La proyección con $\sigma_{zz} = 0$ acoplada a flujo dilatante es notoriamente delicada y queda *out-of-scope* en esta entrega.

```
materials:
- id: 4
  type: DruckerPrager2D
```

```

E: 30.0e9
nu: 0.25
cohesion: 2.0e5      # Pa
phi_deg: 30.0
psi_deg: 10.0      # no asociada (psi < phi)
H: 0.0             # plasticidad perfecta
variant: plane_strain_matched
density: 2000.0     # arena densa

```

5.3 Daño Mecánico

5.3.1 IsotropicDamage1D — daño isótropo 1D con softening exponencial

Daño escalar $d \in [0,1)$ con esfuerzo nominal $\sigma = (1 - d) E \varepsilon$. Evolución por norma de la deformación equivalente $\varepsilon_{eq} = |\varepsilon|$ y umbral κ_0 :

$$d = 1 - \frac{\kappa_0}{\kappa} \exp(-\alpha(\kappa - \kappa_0)) \quad \text{para } \kappa > \kappa_0$$

- **Parámetros:** E, kappa_0 (umbral elástico), alpha (velocidad de softening), density (opcional).
- **Variables internas:** κ (máxima histórica), d (daño).
- **Tangente: algorítmica consistente** en carga activa con daño no saturado: $E_{\tan} = -(1 - d) E \alpha \kappa$ — negativa, refleja la pendiente descendente σ - ε post-pico. En descarga, sin daño activo o al saturar, vuelve a la secante $(1 - d)E$.

5.3.2 IsotropicDamage2D — daño isótropo 2D

Extensión 2D del modelo anterior. Esfuerzo nominal $\sigma = (1 - d) \mathbf{C}_e \varepsilon$. Deformación equivalente simétrica:

$$\varepsilon_{eq} = \sqrt{\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \frac{1}{2}\gamma_{xy}^2}$$

- **Parámetros:** E, nu, kappa_0, alpha, hypothesis (plane_stress default o plane_strain), density (opcional).
- **Tangente algorítmica consistente:** $\mathbf{C}_{\text{alg}} = (1 - d) \mathbf{C}_e - \frac{(1-d)(1/\kappa+\alpha)}{\varepsilon_{eq}} (\mathbf{C}_e \varepsilon) \otimes (\mathbf{M} \varepsilon)$ con $\mathbf{M} = \text{diag}(1,1,1/2)$ en carga activa. **No simétrica** ($\mathbf{C}_e \varepsilon$ y $\mathbf{M} \varepsilon$ no son proporcionales); el atributo IS_SYMMETRIC = False hace que el despachador algebraico [ADR 0003](#) elija LU. Recupera convergencia cuadrática del Newton global frente a la secante.
- **Limitación:** ε_{eq} simétrica no distingue daño en tensión vs compresión; para hormigón se requiere modelo de Mazars o split tensión/compresión (no implementado). Sin regularización por longitud característica $\rightarrow$ *mesh-dependency* en régimen de ablandamiento.

```
materials:
- id: 3
  type: IsotropicDamage2D
  E: 30.0e9
  nu: 0.2
  kappa_0: 1.5e-4
  alpha: 800.0
  hypothesis: plane_stress
  density: 2500.0 # hormigón
```

5.4 Elasticidad Unilateral (Cable)

5.4.1 CableMaterial1D — cable lineal en tensión, nulo en compresión

Material 1D *memoryless* que captura la propiedad esencial del cable: solo transmite esfuerzo cuando está tensado.

$$\sigma(\varepsilon) = E \langle \varepsilon \rangle^+ = \begin{cases} E\varepsilon & \varepsilon > 0 \\ 0 & \varepsilon \leq 0 \end{cases}$$

con $\langle \cdot \rangle^+$ el operador de MacAuley (parte positiva). Tangente discontinua en $\varepsilon = 0$ (asignación al régimen compresivo por convención).

- **Parámetros:** E (módulo de Young en tensión).
- **Sin variables internas** (la respuesta depende sólo de ε instantánea).
- **Caveats numéricos:** en transiciones tensado $\leftrightarrow$ destensado Newton-Raphson puede oscilar. Mitigación: usar `ArcLengthSolver` o pasos finos con `adaptive`.

```
materials:
- id: 1
  type: CableMaterial1D
  E: 150.0e9
  density: 7850.0
```

5.5 Materiales 3D

Los materiales 3D operan sobre la convención **Voigt 6D** del proyecto ([ADR 0012](#)):

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]^T, \quad \boldsymbol{\sigma} = [\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{yz}, \sigma_{xz}]^T$$

con $\gamma_{ij} = 2\varepsilon_{ij}$ (deformación angular *engineering*). Sólo son compatibles con elementos 3D (STRAIN_DIM = 6).

Ninguno admite hypothesis: las variantes `plane_stress` / `plane_strain` no tienen sentido en 3D, donde todas las componentes están activas. Declararla es un error.

Caveat de interoperabilidad con ABAQUS: ABAQUS ordena Voigt como [11, 22, 33, 12, 13, 23], es decir con el bloque cortante permutado ($yz \leftrightarrow xz$) respecto al proyecto. Importar tensores de ABAQUS exige intercambiar las componentes 5 y 6 una sola vez en el preprocesador.

5.5.1 Elastic3D — elástico lineal isótropo 3D

$\sigma = \mathbf{C} \epsilon$ con $\mathbf{C}$ isotrópica 6×6 . Tangente constante, sin variables internas.

- **Parámetros:** E (> 0), $\nu \in (-1, 0.5)$, **density** (opcional).

5.5.2 VonMises3D — plasticidad J2 3D con endurecimiento isótropo lineal

Extensión 3D de `VonMises2D`. Flujo asociado y *return mapping* radial cerrado sobre la parte desviadora (Simó-Hughes §3.3), con tangente algorítmica consistente.

- **Parámetros:** E , ν , **sigma_y** (> 0), H (≥ 0 , default 0), **density** (opcional).
- **Variables internas:** **eps_p** (6 componentes) y **alpha** (deformación plástica equivalente).
- **Relación con el 2D:** bajo la restricción $\epsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ se reduce exactamente a `VonMises2D` en **plane_strain**, verificado a 10 decimales.

5.5.3 DruckerPrager3D — plasticidad friccional 3D

Cono circular suave de Mohr-Coulomb, criterio $f = \sqrt{J_2} + \eta_f I_1 - k(\alpha) \leq 0$ con endurecimiento isótropo lineal en la cohesión. Dos ramas cerradas con detección automática: retorno regular a la superficie y retorno al ápice hidrostático.

- **Parámetros:** E , ν , **cohesion**, **phi_deg** (ángulo de fricción en grados), **psi_deg** (dilatancia en grados, default $= \phi$), H (default 0), **variant** (default **outer_cone**), **density** (opcional).
- **Calibraciones:** **outer_cone** (default, circumscribe Mohr-Coulomb) e **inner_cone** (inscribe). La variante **plane_strain_matched** del modelo 2D **no existe en 3D** y el constructor la rechaza con mensaje explícito: es una calibración 2D por construcción, porque Mohr-Coulomb en 3D depende del ángulo de Lode.
- **Plasticidad no asociada:** con $\psi \neq \phi$ la tangente es asimétrica y el despachador algebraico selecciona LU automáticamente.

5.5.4 IsotropicDamage3D — daño isótropo 3D con softening exponencial

$\sigma = (1 - d) \mathbf{C}_e \epsilon$ con daño escalar d gobernado por la deformación equivalente

$$\epsilon_{eq} = \sqrt{\epsilon_{xx}^2 + \epsilon_{yy}^2 + \epsilon_{zz}^2 + \frac{1}{2}(\gamma_{xy}^2 + \gamma_{yz}^2 + \gamma_{xz}^2)}$$

que es la norma de Frobenius del tensor de deformación y extiende de forma natural la convención 2D. Historial κ máximo con condiciones de Kuhn-Tucker y ley de softening exponencial compartida con las versiones 1D y 2D.

- **Parámetros:** E , ν , **kappa_0** (umbral de daño), **alpha** (parámetro de la ley de softening), **density** (opcional).
- **Tangente:** algorítmica consistente en carga activa (no simétrica en general), secante en descarga y al saturar.


```
materials:
- {id: 1, type: Elastic3D, E: 210.0e9, nu: 0.3, density: 7850.0}
- {id: 2, type: VonMises3D, E: 210.0e9, nu: 0.3, sigma_y: 250.0e6, H: 1.0e9}
```

5.6 Materiales Cohesivos (salto de desplazamientos)

Familia **paralela** al resto del catálogo: no relacionan σ con ε sino la **tracción** $\mathbf{t}$ con el **salto de desplazamientos** $\llbracket u \rrbracket$ a través de una discontinuidad. Viven en su propio registro y se declaran en el bloque `cohesive_materials` del YAML, no en `materials`. Sólo los consumen elementos con discontinuidad embebida como `CST_Embedded2D`.

5.6.1 CohesiveDamageIsotropic — daño cohesivo Modo-I

Daño escalar $\omega \in [0, 1]$ sobre el salto normal: $t_n = (1 - \omega)K_e \llbracket u_n \rrbracket$, con $t_s = 0$ (Modo-I puro). La energía de fractura G_f cierra la curva analíticamente, en variante lineal (apertura crítica $w_c = 2G_f/\sigma_{t0}$) o exponencial (asintótica).

- **Parámetros:** `sigma_t0` (resistencia a tracción, Pa), `G_f` (energía de fractura, N/m), `K_e` (rigidez del salto, Pa/m), `softening` $\in$ `linear`, `exponential`.
- **Sobre `K_e`:** es un *penalty* que representa la cohesión antes de agrietar, **no** una propiedad del bulk. No tiene default automático; como guía, $K_e \approx 10 E_{bulk}/\ell_c$. Valores muy altos endurecen el sistema y dificultan la convergencia; valores bajos introducen flexibilidad espuria antes de la activación.
- **Variables internas:** `kappa` (historial del salto) y `damage` (ω).

```
cohesive_materials:
- {id: 1, type: CohesiveDamageIsotropic, sigma_t0: 2.5e6, G_f: 100.0,
  K_e: 1.0e13, softening: linear}
```

Capítulo 6

Esquemas de Solución

Este capítulo cubre los **solvers estáticos** del catálogo: lineal, no lineal (Newton-Raphson) y arc-length (Crisfield cilíndrico). Los solvers de análisis modal, dinámico (Newmark, HHT- α , central differences) y de análisis en frecuencia / espectral (Harmonic, Response Spectrum) se documentan en el siguiente capítulo, "Análisis Dinámico".

6.1 LinearSolver — solución directa lineal

Resuelve $\mathbf{K} \cdot \mathbf{U} = \mathbf{F}$ con una sola factorización del sistema (el backend lo elige Solidum: Cholesky, Pardiso o LU según la matriz y lo instalado). Las condiciones de Dirichlet se imponen por eliminación directa ([ADR 0004](#)): el sistema entregado al backend algebraico es ya el reducido $\mathbf{K}_{\text{red}} \cdot \mathbf{U}_{\text{libre}} = \mathbf{F}_{\text{red}}$.

- **Cuándo usarlo:** problemas estrictamente lineales (todos los materiales con tangente constante, sin grandes desplazamientos, sin contacto).
- **Parámetros:** `linear_algebra` (default `auto`; ver *Solver* en el capítulo del YAML).
- **Comprobaciones automáticas** ([ADR 0019](#)): antes de resolver, que el modelo esté suficientemente apoyado; después, que la solución cumpla el equilibrio y que la matriz no sea singular. Si algo falla, el análisis se detiene con una explicación en vez de devolver resultados absurdos (ver *Diagnóstico de Problemas*).
- **No aplica:** cualquier no-linealidad material (daño, plasticidad, cable) o geométrica (corotacional). Producirá resultados inconsistentes.

```
solver:  
  type: LinearSolver
```

6.2 NonlinearSolver — Newton-Raphson incremental con paso adaptativo

Bucle externo de incrementos del factor de carga $\lambda \in [0, 1]$; bucle interno Newton-Raphson sobre $\mathbf{K}_t \cdot \Delta \mathbf{U} = \mathbf{R} = \lambda \mathbf{F}_{\text{ext}} - \mathbf{F}_{\text{int}}$.

Criterio de convergencia dual:

$$\max(\text{err}_{\text{disp}}, \text{err}_{\text{force}}) < \text{tol}$$

con $\text{err}_{\text{disp}} = \|\Delta \mathbf{U}\| / (\|\mathbf{U}\| + \epsilon)$ y $\text{err}_{\text{force}} = \|\mathbf{R}\| / \max(\|\lambda \mathbf{F}_{\text{ext}}\|, \|\mathbf{F}_{\text{int}}\|, \epsilon)$.

Adaptatividad (`adaptive: true`):

- Si converge en menos de 5 iteraciones, agranda el siguiente paso ($\times 1.5$).
- Si no converge, biseca el paso ($\div 2$). Falla definitivamente si $\Delta\lambda < \text{min_delta_lambda}$.

```

solver:
  type: NonlinearSolver
  num_steps: 20
  max_iter: 15
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5

```

Parámetros: `convergence` (bloque con `rtol_force`, `rtol_disp`, `atol_force_factor`, `atol_disp_factor`; ver [ADR 0007](#)), `max_iter`, `num_steps`, `adaptive`, `min_delta_lambda`, `linear_algebra`, `freeze_tangent_after_iter`.

Limitación: no puede atravesar puntos límite (snap-through) ni recorrer ramas con derivada negativa de la curva carga-desplazamiento. Para esos casos $\rightarrow$ `ArcLengthSolver`.

6.3 ArcLengthSolver — longitud de arco cilíndrico (Crisfield)

Traza curvas de equilibrio con snap-through, snap-back o pérdida de unicidad de carga, controlando simultáneamente desplazamientos y factor de carga λ .

Esquema:

- **Predictor tangente:** $\Delta \mathbf{U}_t = \mathbf{K}_t^{-1} \cdot \mathbf{F}_{\text{ext}}^{\text{ref}}$; $\Delta\lambda = \pm dl / \|\Delta \mathbf{U}_t\|$. Signo elegido por proyección con el incremento previo.
- **Corrector iterativo:** en cada iteración resuelve dos sistemas ($\mathbf{K} \cdot \Delta \mathbf{U}_R = \mathbf{R}$ y $\mathbf{K} \cdot \Delta \mathbf{U}_t = \mathbf{F}_{\text{ext}}$) y aplica la restricción cuadrática cilíndrica $\|\delta \mathbf{U}\|^2 = dl^2$.
- **Auto-ajuste de dl:** se agranda en convergencias rápidas, se reduce en lentas, biseca al fallar.
- **Caso especial final_step:** si el predictor sobrepasaría `max_lambda`, fija λ a ese valor y resuelve solo desplazamientos (Newton-Raphson puro).

```

solver:
  type: ArcLengthSolver
  max_iter: 15
  max_lambda: 1.0
  initial_dl: 0.05
  max_steps: 200
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5

```

Cuándo usarlo: problemas con softening pronunciado (daño, post-pandeo, snap-through de cúpulas), o cuando `NonlinearSolver` diverge cerca de un punto límite. Más caro por paso (dos `spsolve` por iteración) pero indispensable en estos casos.

Referencia: Crisfield, "A fast incremental/iterative solution procedure that handles snap-through" (*Computers & Structures*, 1981).

6.4 DissipationArcLengthSolver — arc-length por disipación de energía

Variante de `ArcLengthSolver` que controla la **disipación incremental de energía** por paso en lugar de la longitud de arco euclídea. Es la herramienta indicada cuando la curva $\mathbf{U}$ - λ tiene tramos casi verticales, donde la restricción cilíndrica se vuelve mal condicionada.

Diferencia esencial: la restricción de paso

$$g(\Delta \mathbf{U}, \Delta \lambda) = \frac{1}{2} (\lambda_n \mathbf{F} \cdot \Delta \mathbf{U} - \Delta \lambda \mathbf{F} \cdot \mathbf{U}_n) = \tau$$

es **lineal** en $(\Delta \mathbf{U}, \Delta \lambda)$, frente a la cuadrática del cilíndrico. El corrector tiene por tanto una sola raíz: desaparecen la selección de raíz por menor ángulo y la patología de raíces imaginarias.

- **Switching automático:** arranca en modo cilíndrico (la restricción por disipación es idénticamente nula mientras $\lambda_n = \mathbf{U}_n = 0$) y conmuta a disipación en cuanto detecta disipación neta sobre el umbral. La conmutación es automática en ambos sentidos.
- **Parámetros:** todos los de `ArcLengthSolver`, más `initial_tau` (obligatorio) y la familia `tau_grow_factor`, `tau_max_factor`, `tau_shrink_factor`, `tau_grow_iter_threshold`, `tau_shrink_iter_threshold`, `dissipation_threshold`.
- **Cuándo usarlo:** softening de daño continuo (1D/2D) con rama post-pico pronunciada, donde el cilíndrico converge mal.
- **Limitación conocida:** no resuelve el caso de discontinuidad embebida con penalty cohesivo rígido. La activación discreta del criterio de Rankine produce un salto en las fuerzas internas que el seguimiento de signo aproximado no maneja. Es una limitación documentada del solver, no un error de configuración.

```
solver:
type: DissipationArcLengthSolver
max_lambda: 1.0
initial_dl: 0.05
initial_tau: 1.0e-4
max_steps: 300
```

Referencia: Gutiérrez, "Energy release control for numerical simulations of failure in quasi-brittle solids" (*Communications in Numerical Methods in Engineering*, 2004); switching a la Verhoosel et al. (2009).

Capítulo 7

Análisis Dinámico

Solidum FEM expone el subsistema modal/dinámico/espectral **completo** ([ADR 0009](#) cerrado en su totalidad 2026-05-18). Cubre los siete regímenes canónicos de la mecánica computacional clásica:

1. **Modal**: autovalores generalizados $\mathbf{K}\phi = \omega^2\mathbf{M}\phi$ (`ModalSolver`).
2. **Mass lumping**: matriz de masa diagonal vía HRZ canónico (sólidos) o nodal directo (frames). Disponible en todos los elementos vía `compute_mass_matrix(lumping="lumped")`.
3. **Transitorio implícito lineal**: Newmark- β (`NewmarkSolver`) y variante HHT- α con disipación numérica (`HHTSolver`).
4. **Transitorio implícito no lineal**: Newton-Newmark (`NewtonNewmarkSolver`) y Newton-HHT (`NewtonHHTSolver`).
5. **Transitorio explícito**: diferencias centradas leapfrog (`CentralDifferenceSolver`).
6. **Frecuencia**: respuesta forzada armónica (`HarmonicSolver`).
7. **Espectral / sísmico**: combinación modal SRSS/CQC contra espectros normativos (`ResponseSpectrumSolver`).

Todos los solvers consumen la masa global que cada elemento devuelve por `compute_mass_matrix(lumping)` con `lumping` $\in$ {"consistent", "lumped"} y la densidad declarada en el material (`density`; obligatoria para cualquier análisis dinámico, [ADR 0008](#)).

7.1 ModalSolver — análisis modal por autovalores generalizados

Resuelve el problema generalizado $\mathbf{K}\phi = \omega^2\mathbf{M}\phi$ con $\mathbf{K}$ evaluada en $\mathbf{u} = 0$ (régimen lineal) y $\mathbf{M}$ ensamblada por cuadratura consistente sobre todos los elementos. La capa algebraica (`EigenSolver`) envuelve `scipy.sparse.linalg.eigsh` con *shift-invert* en σ (ARPACK Lanczos), de modo que basta una sola factorización de $\mathbf{K} - \sigma\mathbf{M}$ para extraer las n frecuencias más bajas.

Parámetros: `n_modes` (obligatorio), `sigma` (default 0.0; `sigma=0` extrae las frecuencias más bajas), `which` (default "LM"), `tolerance` (default 1.0e-9), `lumping` (default "consistent"; la masa concentrada queda diferida a fase 2 del [ADR 0009](#)).

Salida: instancia de `ModalResult` con `frequencies_rad`, `frequencies_hz`, `periods`, `modes` (M-ortonormales). El método `ModalResult.free_vibration(M, u0, u0_dot, t)` reconstruye analíticamente la respuesta libre no amortiguada por superposición modal.

```
nodes:
- {id: 1, coords: [0.0, 0.0]}
- {id: 2, coords: [1.0, 0.0]}

materials:
- {id: 1, type: Elastic1D, E: 200.0e9, density: 7850.0}

elements:
- {id: 1, type: Truss2D, material: 1, A: 0.01, nodes: [1, 2]}

boundary_conditions_by_node:
- {node_id: 1, ux: 0.0, uy: 0.0}
- {node_id: 2, uy: 0.0}

solver:
type: ModalSolver
n_modes: 3
sigma: 0.0
```

Nota

La densidad es *obligatoria* para todo material involucrado en análisis modal o transitorio. La omisión lanza `ValueError` ([ADR 0008](#)) listando los materiales afectados por nombre antes de iniciar el cálculo.

7.2 NewmarkSolver — transitorio lineal Newmark- β

Integra en el tiempo $\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}(t)$ por la familia Newmark- β . Con $(\beta, \gamma) = (1/4, 1/2)$ — *average acceleration*, default — es *incondicionalmente estable*, sin amortiguamiento numérico, con error $\mathcal{O}(\Delta t^2)$.

Esquema operativo:

- Predictor: $\tilde{\mathbf{u}} = \mathbf{u}_n + \Delta t \dot{\mathbf{u}}_n + (\Delta t^2/2)(1 - 2\beta)\ddot{\mathbf{u}}_n$ y análogo para $\tilde{\dot{\mathbf{u}}}$.
- Sistema efectivo en aceleración: $\mathbf{A}_{\text{eff}}\ddot{\mathbf{u}}_{n+1} = \mathbf{F}_{n+1} - \mathbf{C}\tilde{\dot{\mathbf{u}}} - \mathbf{K}\tilde{\mathbf{u}}$ con $\mathbf{A}_{\text{eff}} = \mathbf{M} + \gamma\Delta t \mathbf{C} + \beta\Delta t^2 \mathbf{K}$. Factorización *única* al inicio del análisis y reusada en todos los pasos (`FactorizedSolver`, [ADR 0003](#) fase 2).
- Corrector: $\mathbf{u}_{n+1} = \tilde{\mathbf{u}} + \beta\Delta t^2 \ddot{\mathbf{u}}_{n+1}$, $\dot{\mathbf{u}}_{n+1} = \tilde{\dot{\mathbf{u}}} + \gamma\Delta t \ddot{\mathbf{u}}_{n+1}$.

Amortiguamiento Rayleigh $\mathbf{C} = \alpha\mathbf{M} + \beta\mathbf{K}$, declarado de dos formas:

- Directa: `rayleigh: {alpha: 0.1, beta: 1.0e-4}`.
- Calibración modal: `rayleigh: {xi1: 0.02, omega1: 50.0, xi2: 0.02, omega2: 200.0}` — Solidum resuelve los (α, β) que reproducen los amortiguamientos ξ_1, ξ_2 a las frecuencias ω_1, ω_2 .

Parámetros: `t_end`, `dt` (obligatorios); `beta` (default 0.25), `gamma` (default 0.5), `rayleigh` (opcional), `u0`, `u0_dot` (condiciones iniciales, default ceros), `F_func` (callback Python $t \rightarrow \mathbf{F}(t)$; default vibración libre). Desde YAML, las cargas del archivo (puntuales + peso propio / fuerza de cuerpo) se inyectan automáticamente como escalón constante en el tiempo; para una historia $\mathbf{F}(t)$ arbitraria, pasar `F_func` desde Python.

Salida: `TransientResult` con `t_history`, `u_history`, `udot_history`, `uddot_history` (forma $(n_{\text{dof}}, n_{\text{steps}} + 1)$) y los $\alpha_{\text{Rayleigh}}, \beta_{\text{Rayleigh}}$ efectivos. `result.internal_forces_history(domain)` da las fuerzas internas (N, V, M, T) de los elementos estructurales 1D en cada paso: con `record_internal_forces: true` (parámetro común a los cinco solvers transitorios, también desde YAML) se registran **durante** el análisis con el estado interno de cada paso y quedan en `result.element_forces_history`; sin él se reconstruyen a posteriori desde `u_history` con el estado final, lo que sólo es exacto sin historia (materiales elásticos, elementos geoméricamente lineales) y avisa por log en caso contrario.

```
solver:
  type: NewmarkSolver
  t_end: 2.0
  dt: 0.005
  beta: 0.25
  gamma: 0.5
  rayleigh:
    xi1: 0.02
    omega1: 50.0
    xi2: 0.02
    omega2: 200.0
```

Limitaciones: lineal ($\mathbf{K}, \mathbf{M}$ constantes; para no-linealidad material o geométrica $\rightarrow$ `NewtonNewmarkSolver`). Apoyos Dirichlet constantes en el tiempo. Paso Δt fijo (adaptativo diferido). HHT- α y generalized- α (con amortiguamiento numérico controlado) no incluidos.

7.3 NewtonNewmarkSolver — transitorio no lineal Newton-Newmark

Subclase de `NewmarkSolver` (Reglas §4 — variante de componente existente) que añade un bucle Newton-Raphson dentro de cada paso temporal sobre el residuo dinámico:

$$\mathbf{R} = \mathbf{F}_{\text{ext}}(t) - \mathbf{F}_{\text{int}}(\mathbf{u}) - \mathbf{C}\dot{\mathbf{u}} - \mathbf{M}\ddot{\mathbf{u}}, \quad \mathbf{J} = \mathbf{M} + \gamma\Delta t \mathbf{C} + \beta\Delta t^2 \mathbf{K}_t$$

Convergencia dual ([ADR 0007](#)): mismas tolerancias `rtol_force` / `rtol_disp` y factores `atol_*_factor` que los solvers no lineales estáticos. Tras converger cada paso se invoca `assembler.commit_all_states()` (`trial` $\rightarrow$ `committed` en todos los elementos); si agota `max_iter` se lanza `RuntimeError` y se descarta el estado `trial`.

Amortiguamiento Rayleigh constante en el tiempo, calibrado con la rigidez elástica de referencia $\mathbf{K}_0$ al inicio del análisis. Convención estándar (Abaqus, ANSYS, OpenSees) que evita acoplamiento ad-hoc entre disipación viscosa y plástica.

Newton modificado opcional (`freeze_tangent_after_iter`, [ADR 0003](#) fase 2): factoriza fresco las primeras N iter y reusa la factorización en las siguientes para abaratar pasos largos.

Recuperación del caso lineal: con materiales sin historia o no plastificados, el residuo se anula en 1 iter y el resultado coincide *a paridad de bits* con `NewmarkSolver`. Validado en tests.

Parámetros: heredados de `NewmarkSolver` más `max_iter` (default 20), bloque `convergence` (`rtol_force`, `rtol_disp`, `atol_force_factor`, `atol_disp_factor`; [ADR 0007](#)) y `freeze_tangent_after_iter` (default `None`). El despacho YAML es automático: como `NewtonNewmarkSolver` es subclase de `NewmarkSolver`, `solidum.run_yaml` lo detecta vía `isinstance` y enruta a `run_transient`.

```
materials:
- {id: 1, type: VonMises2D, E: 200.0e9, nu: 0.3, sigma_y: 250.0e6, H: 2.0e9,
  hypothesis: plane_strain, density: 7850.0}

solver:
  type: NewtonNewmarkSolver
  t_end: 0.5
  dt: 0.001
  beta: 0.25
  gamma: 0.5
  rayleigh:
    xi1: 0.02
    omega1: 100.0
    xi2: 0.02
    omega2: 500.0
  max_iter: 15
  convergence:
    rtol_force: 1.0e-6
    rtol_disp: 1.0e-6
```

Cuándo usarlo: respuesta dinámica de estructuras con plasticidad transitoria (sísmica con disipación plástica, impacto en hormigón friccional, fatiga de bajo ciclo), vibraciones de marcos elastoplásticos. Materiales con historia disponibles en dinámica: `Elastoplastic1D`, `VonMises2D` (*plane strain* y *plane stress*), `DruckerPrager2D`, `IsotropicDamage1D/2D`.

Limitaciones: igual que `NewmarkSolver` en lo lineal (apoyos constantes, paso fijo). No resuelve *snap-back* dinámico con softening severo — combinar con `ArcLengthSolver` si emerge.

7.4 HHTSolver y NewtonHHTSolver — variantes Hilber-Hughes-Taylor

Variantes de Newmark con **disipación numérica controlada** en altas frecuencias (Hilber, Hughes & Taylor 1977). Útiles cuando modos altos espurios contaminan la respuesta y conviene atenuarlos preservando segundo orden y estabilidad incondicional.

El parámetro $\alpha \in [-1/3, 0]$ controla la disipación; los coeficientes (β, γ) se auto-derivan canónicamente:

$$\beta = \frac{(1 - \alpha)^2}{4}, \quad \gamma = \frac{1 - 2\alpha}{2}$$

con radio espectral $\rho_\infty = (1 + \alpha)/(1 - \alpha)$. Recuperación exacta a Newmark cuando $\alpha = 0$.

Esquema operativo: idéntico a Newmark pero con factor $(1 + \alpha)$ en los términos elásticos y de amortiguamiento del residuo dinámico, y $-\alpha$ multiplicando los del paso anterior. Variantes lineal (`HHTSolver`) y no lineal (`NewtonHHTSolver`) heredan toda la maquinaria de `NewmarkSolver` / `NewtonNewmarkSolver` (Reglas §4 sobre variantes).

```
solver:
  type: HHTSolver                # o NewtonHHTSolver para materiales con historia
  t_end: 2.0
```



```
dt: 0.005
alpha: -0.05 # default; disipación ligera de altas frecuencias
```

7.5 CentralDifferenceSolver — explícito por diferencias centradas (ADR 0009 fase 5)

Integración **explícita** de segundo orden por leapfrog Belytschko-Liu-Moran. Sin sistema lineal en cada paso — la única "inversión" es $\mathbf{M}^{-1}$, trivial con masa lumped diagonal. Apropiado para wave propagation, impacto, dinámica de respuesta rápida.

Esquema operativo (forma predictor-corrector):

$$\mathbf{a}_0 = \mathbf{M}^{-1}(\mathbf{F}(0) - \mathbf{F}_{\text{int}}(\mathbf{u}_0) - \mathbf{C}\mathbf{v}_0); \quad \mathbf{v}_{1/2} = \mathbf{v}_0 + \frac{\Delta t}{2}\mathbf{a}_0$$

Paso $n \rightarrow n + 1$:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t \mathbf{v}_{n+1/2}; \quad \mathbf{a}_{n+1} = \mathbf{M}^{-1}(\mathbf{F}_{n+1} - \mathbf{F}_{\text{int}}(\mathbf{u}_{n+1}) - \mathbf{C}\mathbf{v}_{n+1/2})$$

$$\mathbf{v}_{n+1} = \mathbf{v}_{n+1/2} + \frac{\Delta t}{2}\mathbf{a}_{n+1}$$

Una sola clase cubre lineal y no lineal con parámetro **nonlinear**: con **False** usa $\mathbf{F}_{\text{int}} = \mathbf{K}\mathbf{u}$ con $\mathbf{K}$ constante; con **True** recalcula $\mathbf{F}_{\text{int}}$ cada paso vía `Assembler.assemble_non_linear_system`.

Condición de estabilidad CFL: $\Delta t < 2/\omega_{\text{máx}}$. Para barras: $\Delta t < L_{\text{el}}\sqrt{\rho/E}$. El solver detecta divergencia exponencial a posteriori y aborta con `RuntimeError` informativo si la condición se viola.

Requisitos: `lumping="lumped"` obligatorio ("consistent" rechazado con `ValueError` — el coste de invertir $\mathbf{M}$ consistente cada paso anula la ventaja del explícito). `Frame3D` con eje oblicuo a los ejes globales también rechazado (la rotación rompe la diagonalidad estricta — limitación documentada estándar, Cook-Malkus-Plesha §11.4).

```
solver:
type: CentralDifferenceSolver
t_end: 2.5
dt: 0.005 # debe respetar CFL: Δt < L_el·√ρ( /E)
lumping: lumped # obligatorio
nonlinear: false # true para materiales con historia
u0: [0.0, 0.0, 1.0, 0.0]
```

7.6 HarmonicSolver — respuesta forzada armónica en frecuencia (ADR 0009 fase 6)

Análisis lineal en estado estacionario: dada una excitación armónica $\mathbf{F}(t) = \text{Re}\{\hat{\mathbf{F}}e^{i\omega t}\}$, calcula la amplitud compleja $\hat{\mathbf{u}}(\omega)$ de la respuesta estacionaria $\mathbf{u}(t) = \text{Re}\{\hat{\mathbf{u}}(\omega)e^{i\omega t}\}$ para un barrido de frecuencias.

Ecuación resuelta por frecuencia:

$$(-\omega^2\mathbf{M} + i\omega\mathbf{C} + \mathbf{K})\hat{\mathbf{u}}(\omega) = \hat{\mathbf{F}}$$

Aritmética compleja directa con `scipy.sparse.linalg.spsolve`. Factorización LU compleja por frecuencia (no se cachea — la matriz cambia con ω). Coste dominante: $O(n_\omega \cdot \text{coste_factorizacion})$.

Barrido configurable:

- Vector explícito vía `omega` (lista de rad/s).
- Lineal: `omega_min`, `omega_max`, `n_omega`, `scale: linear`.
- Logarítmico: `scale: log` con `np.geomspace`.

Cargas: las `point_loads` del bloque YAML estándar se inyectan automáticamente como `F_amplitude` real con fase cero. Cargas con fase compleja requieren construcción desde código Python (YAML no soporta tipos complejos nativos).

Salida: `HarmonicResult` con `omega`, `u_complex` (shape $(n_{\text{dof}}, n_\omega)$), `F_complex`, y métodos `.amplitude()` ($|\hat{u}|$) y `.phase()` ($\arg \hat{u}$). Propiedad derivada `frequencies_hz`.

```
point_loads:
- {node_id: 2, ux: 1.0}      # amplitud real (fase 0)

solver:
  type: HarmonicSolver
  omega_min: 1.0
  omega_max: 10.0
  n_omega: 91
  scale: linear              # o "log" para FRFs sobre rangos amplios
  rayleigh:
    alpha: 0.5
    beta: 0.0
```

Cuándo usarlo: funciones de transferencia (FRFs), diagnóstico de resonancias antes de un transitorio caro, respuesta forzada por maquinaria rotante o cargas armónicas descompuestas.

Caveats: cerca de resonancia exacta sin amortiguamiento $\mathbf{Z}(\omega_n)$ es singular y el `spsolve` puede fallar — añadir Rayleigh.

7.7 ResponseSpectrumSolver — análisis sísmico por combinación modal (ADR 0009 fase 7)

Análisis sísmico clásico: el suelo se caracteriza por un espectro de respuesta (en aceleración $S_a(T)$ o desplazamiento $S_d(T)$ vs período), no por un acelerograma temporal. El solver combina las respuestas máximas modales en una envolvente máxima — pierde la fase temporal entre modos por construcción, sólo conserva amplitudes envolventes.

Esquema operativo:

1. Análisis modal interno vía `ModalSolver(n_modes, sigma, lumping)`.
2. Factores de participación $\Gamma_n = \phi_n^T \mathbf{M} \mathbf{r}$ con $\mathbf{r}$ el vector de excitación rígida (dirección sísmica).
3. Respuesta máxima por modo: $\mathbf{u}_n^{\text{máx}} = \Gamma_n \phi_n S_d(\omega_n)$.
4. **Combinación SRSS** (Square Root of Sum of Squares, modos bien separados):

$$u_k^{\text{máx}} = \sqrt{\sum_n (u_{k,n}^{\text{máx}})^2}$$

1. **Combinación CQC** (Complete Quadratic Combination, Der Kiureghian 1980, para modos cercanos):

$$u_k^{\text{máx}} = \sqrt{\sum_i \sum_j \rho_{ij} u_{k,i}^{\text{máx}} u_{k,j}^{\text{máx}}}, \quad \rho_{ij} = \frac{8\xi^2(1+r)r^{3/2}}{(1-r^2)^2 + 4\xi^2 r(1+r)^2}$$

con $r = \omega_i/\omega_j$. Cuando $\xi \rightarrow 0$ o $r \rightarrow 0$: $\rho_{ij} \rightarrow \delta_{ij}$ y CQC degenera a SRSS.

Interfaz del espectro:

- Callable directo `spectrum(omega) -> S_d`.
- Dict tabulado: `{type: tabulated, kind: Sd|Sa, periods: [...], values: [...]}`. Interpolación lineal en período; clamp fuera de rango.
- Constante: `{type: constant_acceleration, Sa: ...}`.

Dirección de excitación:

- Vector explícito (`np.ndarray` shape $(n_{\text{dof}},)$).
- Atajo por DOF: `{dof_name: ux}` — el solver construye el vector unitario en ese DOF de todos los nodos.

```

solver:
  type: ResponseSpectrumSolver
  n_modes: 5
  direction:
    dof_name: ux
  spectrum:
    type: tabulated
    kind: Sa                      # aceleración espectral; el solver convierte a Sd
    periods: [0.1, 0.5, 1.0, 2.0, 5.0]
    values: [10.0, 10.0, 5.0, 1.0, 0.25]
  combination: SRSS              # o "CQC" para modos cercanos
  damping: 0.05                  # ξ- sólo usado por CQC

```

Salida: `ResponseSpectrumResult` con `u_combined` (envolvente, positiva), `u_per_mode` (contribución modal con signo), `participation_factors` (Γ_n), `effective_masses` (Γ_n^2), `frequencies_rad`, `combination`, `damping`, `direction`. Método `.cumulative_effective_mass_ratio()` para verificar que la masa modal acumulada alcanza el objetivo normativo (≥ 0.9).

Cuándo usarlo: diseño sísmico contra espectros normativos (ASCE 7, Eurocódigo 8, NCh 433, NTC-DS México). Diagnóstico de qué modos dominan la respuesta en una dirección de excitación.

Caveats: precisión depende del número de modos incluidos (verifica `cumulative_effective_mass_ratio` ≥ 0.9). Excitación multi-direccional simultánea (CQC3, regla 100/30/30) requiere combinar varios análisis fuera del solver.

7.8 Mass lumping (ADR 0009 fase 2)

Independiente de cualquier solver: todos los elementos del catálogo soportan `compute_mass_matrix(lumping="lumped")`. El esquema se elige por familia:

- **Sólidos isoparamétricos** (Tri3, Quad4, Tri6, Quad8, Quad9): **HRZ canónico** (Hinton-Rock-Zienkiewicz 1976) centralizado en `solidum.math.mass_lumping.lump_hrz`. Preserva masa total y proporcionalidad diagonal; evita masas negativas en elementos de orden superior.
- **Vigas y marcos** (Truss, Cable, Frame2D Euler/Timoshenko/EulerCorot, Frame3D): **lumping nodal directo** — $\rho AL/2$ traslacional + $\rho IL/2$ rotacional por nodo. Único esquema que sobrevive a la rotación local→global manteniendo **M** diagonal.

Diagonalidad en globales: garantizada en todos los elementos excepto Frame3D con eje oblicuo (queda **bloque-diagonal** por nodo — limitación documentada estándar; `CentralDifferenceSolver` rechaza este caso explícitamente).

Capítulo 8

Análisis Térmico

Solidum FEM resuelve **conducción de calor** por la ley de Fourier, en régimen estacionario y transitorio, sobre dominios 2D y 3D. El campo incógnita es un escalar T por nodo.

Alcance actual: conducción pura con condiciones de Dirichlet (temperatura impuesta) y Neumann (flujo prescrito). **No** incluye convección (Robin), radiación, conductividad dependiente de la temperatura $k(T)$, cambio de fase ni **acoplamiento termomecánico** — no existe deformación térmica $\alpha \Delta T$: el análisis térmico y el mecánico son hoy independientes.

8.1 Formulación

La ecuación de balance de energía, con $\mathbf{q} = -\mathbf{k} \cdot \nabla T$ (Fourier):

$$\rho c \dot{T} = \nabla \cdot (\mathbf{k} \nabla T) + Q$$

Discretizada por elementos finitos:

$$\mathbf{C} \dot{\mathbf{T}} + \mathbf{K} \mathbf{T} = \mathbf{F}$$

donde $\mathbf{K} = \int_{\Omega} \mathbf{B}^T \mathbf{k} \mathbf{B} d\Omega$ es la matriz de **conductividad**, $\mathbf{C} = \int_{\Omega} \rho c \mathbf{N}^T \mathbf{N} d\Omega$ la de **capacidad calorífica** y $\mathbf{F}$ recoge la fuente volumétrica y el flujo de frontera.

Nótese que el sistema es de **primer orden** en el tiempo, a diferencia del mecánico $\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}$. No hay aceleración ni inercia: no existe un análogo térmico de la velocidad con significado físico en este modelo.

Sin notación Voigt. El gradiente ∇T es un vector genuino de 2 ó 3 componentes, no un tensor simétrico comprimido, así que la matriz $\mathbf{B}$ térmica es el gradiente crudo $\partial N / \partial x$ y no lleva los factores $\gamma = 2\varepsilon$ de la notación Voigt mecánica. Los elementos térmicos declaran FLUX_DIM (2 ó 3) en lugar de STRAIN_DIM.

8.2 Material: ThermalConduction

Se declara en un bloque `thermal_materials`, paralelo a `materials`. Los dos bloques tienen espacios de nombres de `type`: independientes, y un elemento resuelve su material contra el bloque donde se declaró el `id`.

Tabla 8.1: Parámetros de `ThermalConduction`.

Parámetro	Unidades	Obligatorio	Descripción
<code>k</code>	W/(m·K)	sí	Conductividad. Escalar (isótropo) o tensor completo
<code>dim</code>	—	no (2)	Dimensión del problema cuando <code>k</code> es escalar
<code>c</code>	J/(kg·K)	sólo transitorio	Calor específico
<code>density</code>	kg/m ³	sólo transitorio	Densidad

```
thermal_materials:
- id: 1
  type: ThermalConduction
  k: 45.0          # acero estructural
  c: 460.0         # sólo hace falta en transitorio
  density: 7850.0  # sólo hace falta en transitorio
```

`c` y `density` son **opcionales**: el régimen estacionario no los necesita, porque el término $\dot{\mathbf{CT}}$ desaparece. Si faltan y se pide un transitorio, el error lo dice explícitamente y sugiere el solver estacionario.

Conductividad anisótropa: `k` admite un tensor completo, sin necesidad de un material distinto. Debe ser simétrico y definido positivo — ambas cosas se validan al construir.

```
thermal_materials:
- id: 1
  type: ThermalConduction
  k: [[45.0, 0.0], [0.0, 12.0]] # 3.75× más conductor en x que en y
```

8.3 Elementos: Quad4Thermal y Hex8Thermal

Un DOF escalar T por nodo. Comparten geometría y numeración de nodos con sus gemelos mecánicos Quad4 y Hex8.

Tabla 8.2: Elementos térmicos.

Elemento	FLUX_DIM	Nodos	Cuadratura default	Parámetros
Quad4Thermal	2	4	2x2	<code>thickness</code> (default 1.0), <code>quadrature</code>
Hex8Thermal	3	8	hex_2x2x2	<code>quadrature</code>

```
elements:
- id: 1
  type: Quad4Thermal
  material: 1          # id del bloque thermal_materials
  nodes: [1, 2, 3, 4]
  thickness: 0.3
```

Pasar un material mecánico a un elemento térmico —o al revés— se rechaza al construir, con un mensaje que nombra las dos familias. La dimensión también se comprueba: un material construido con `dim: 2` no entra en un `Hex8Thermal`.

8.4 Condiciones de frontera

8.4.1 Dirichlet — temperatura impuesta

Se declaran como cualquier otra condición de frontera, usando el nombre del DOF:

```
boundary_conditions:
- {node_id: 1, T: 100.0}
- {node_id: 2, T: 20.0}
```

Todo modelo térmico necesita al menos un Dirichlet. Sin ninguno, la matriz de conductividad es singular: un campo de temperatura uniforme no produce gradiente ni flujo, y el nivel absoluto de temperatura queda indeterminado. Es el análogo térmico de un modo de sólido rígido, y el error lo nombra así en vez de reportar un fallo algebraico genérico.

8.4.2 Neumann — flujo prescrito

```
thermal_loads:
  boundary_flux:
    - {element: 1, edge: 2, q: 500.0}      # 2D: borde del elemento
    - {element: 7, face: 4, q: -200.0}     # 3D: cara del elemento
```

Convención de signo: $\bar{q} > 0$ es flujo **saliente** del dominio, es decir **enfriamiento**. Un flujo entrante (calentamiento) se declara con $\bar{q} < 0$. El signo sale de la forma débil, donde el término de frontera aparece como $-\int_{\Gamma} w \bar{q} d\Gamma$.

Un borde o cara sin declarar es adiabático ($\bar{q} = 0$), igual que un borde sin carga es libre de tracción en mecánica. No hay que declarar el flujo nulo.

Los índices de borde y cara son los mismos que en los elementos mecánicos: `edge: 0` conecta los nodos locales 0-1, `edge: 1` los 1-2, etc.

8.4.3 Fuente volumétrica

```
thermal_loads:
  body_source:
    - {Q: 1.0e5}      # W/m³, a todo el dominio
    - {Q: 5.0e4, elements: [3, 4, 5]} # sólo a esos elementos
```

Q es generación de calor por unidad de volumen (calor de hidratación, efecto Joule, reacción química). Es **carga del elemento y no propiedad del material**: varía en el espacio y con el tiempo, y como propiedad obligaría a duplicar materiales.

También pueden aplicarse fuentes **nodales concentradas** [W] por la vía estándar:

```
point_loads:
- {node_id: 12, T: 250.0}
```

8.5 Régimen estacionario

$\mathbf{KT} = \mathbf{F}$ lo resuelve el `LinearSolver` sin ninguna configuración especial:

```
solver:
  type: LinearSolver
```

El resultado es un `SolveResult` como el de un análisis mecánico; el campo `U` contiene las temperaturas nodales, y `reactions_by_node` los flujos de calor en los nodos con temperatura impuesta.

8.6 Régimen transitorio: ThetaMethodSolver

Integra $\mathbf{C}\dot{\mathbf{T}} + \mathbf{KT} = \mathbf{F}(t)$ ponderando el estado dentro del paso con el parámetro θ :

$$(\mathbf{C} + \theta\Delta t \mathbf{K}) \mathbf{T}_{n+1} = [\mathbf{C} - (1 - \theta)\Delta t \mathbf{K}] \mathbf{T}_n + \Delta t [\theta \mathbf{F}_{n+1} + (1 - \theta)\mathbf{F}_n]$$

```
solver:
  type: ThetaMethodSolver
  dt: 60.0
  n_steps: 500
  T_initial: 20.0
```

Tabla 8.3: Parámetros de `ThetaMethodSolver`.

Parámetro	Default	Descripción
<code>dt</code>	—	Paso temporal [s]. Obligatorio
<code>n_steps</code>	—	Número de pasos. Obligatorio
<code>T_initial</code>	—	Condición inicial: escalar (campo uniforme) o vector. Obligatorio
<code>theta</code>	1.0	Peso del esquema $\in [0, 1]$
<code>lumping</code>	"lumped"	Forma de la matriz de capacidad
<code>output_every</code>	1	Almacenar el campo cada N pasos

No hay condición inicial de velocidad: la ecuación es de primer orden y $\mathbf{T}_0$ determina la evolución por completo. Tampoco existe el parámetro `rayleigh`: el amortiguamiento de Rayleigh modela disipación en la ecuación de segundo orden, y aquí la disipación es la propia conducción, ya contenida en $\mathbf{K}$.

8.6.1 Elección de θ

Estabilidad no significa ausencia de oscilaciones, y esta distinción decide el valor por defecto.

Crank-Nicolson ($\theta = 1/2$) es de segundo orden y por tanto el más preciso, pero su factor de amplificación tiende a -1 para los modos de frecuencia alta en lugar de a 0. Ante un **cambio brusco** —un escalón de temperatura impuesto en la frontera, típico en el arranque— produce oscilaciones amortiguadas lentamente, con temperaturas que pueden **salirse del rango de los datos**. Eso viola

Tabla 8.4: Casos particulares del esquema θ .

θ	Nombre	Orden	Estabilidad
0	Euler explícito	1	Condicional ($\Delta t \leq 2/\lambda_{\text{máx}}$)
0.5	Crank-Nicolson	2	Incondicional (A-estable)
2/3	Galerkin	1	Incondicional
1	Euler implícito	1	Incondicional, L-estable (default)

el *principio del máximo* de la ecuación de difusión, según el cual la solución exacta nunca excede los extremos de los datos iniciales y de frontera: no es un resultado impreciso, es un resultado imposible.

Euler implícito ($\theta = 1$) es sólo de primer orden, pero **L-estable**: amortigua los modos altos por completo en un paso y nunca oscila. Por eso es el default.

En un ensayo con pared a 100 y cuerpo inicialmente a 0, con paso grande, $\theta = 1$ se mantiene en $[0, 100]$ y es monótono, mientras $\theta = 1/2$ alcanza 151.27.

Si su problema es suave y busca la precisión de segundo orden, use `theta: 0.5` conscientemente. El resultado expone el campo `order` (2 con Crank-Nicolson, 1 en el resto) para que el coste en precisión del default robusto sea visible.

8.6.2 Elección de Δt

Con $\theta \geq 0.5$ **ningún paso diverge**, pero eso no significa que cualquier paso sirva. La escala física la da la difusividad $\alpha = k/(\rho c)$ y el tamaño de elemento h :

$$\Delta t_{\text{car}} \sim \frac{h^2}{\alpha}$$

Es el tiempo que tarda el frente térmico en atravesar un elemento. Un Δt mucho mayor es estable pero **se salta el transitorio** que se quiere observar. El solver reporta este valor al arrancar y avisa si el paso elegido lo supera ampliamente. Es información, no restricción.

En el otro extremo, el transitorio no ha **terminado** hasta $t \gg L^2/\alpha$ con L la dimensión global del dominio. Integrar por debajo de esa escala y comparar con el estacionario da diferencias que no son error del esquema, sino un análisis inacabado.

8.6.3 Capacidad lumped por defecto

A diferencia del análisis dinámico estructural, donde el default es **consistent**, aquí la matriz de capacidad se agrupa (**lumped**) por defecto. El motivo es el mismo principio del máximo: la capacidad consistente produce oscilaciones espurias ante un frente térmico abrupto. Los dos defaults invertidos —**lumped** en el eje espacial y $\theta = 1$ en el temporal— atacan el mismo fenómeno desde lados distintos.

8.6.4 Resultado

`ThetaMethodSolver` devuelve un `ThermalTransientResult`, no el `TransientResult` del análisis dinámico: no hay velocidad, aceleración ni coeficientes de Rayleigh que reportar.

Tabla 8.5: Campos de `ThermalTransientResult`.

Campo / método	Descripción
<code>t_history</code>	Instantes almacenados, shape <code>(n_stored,)</code>
<code>T_history</code>	Campo global, shape <code>(n_dof, n_stored)</code>
<code>T_final</code>	Campo en el último instante
<code>temperature_at(dof)</code>	Historia temporal de un DOF concreto
<code>extremes()</code>	<code>(T_min, T_max)</code> sobre todos los nodos e instantes
<code>n_steps, theta, dt, order</code>	Parámetros efectivos del análisis

`extremes()` es la herramienta de diagnóstico del principio del máximo: en un problema sin fuente, cualquier valor fuera del rango de los datos señala oscilación espuria del esquema, no un fenómeno físico.

8.7 Ejemplo completo: pared plana en régimen transitorio

Pared de acero de 2 m, inicialmente a 20 °C, con una cara llevada súbitamente a 100 °C.

```

nodes:
- {id: 1, coords: [0.0, 0.0]}
- {id: 2, coords: [1.0, 0.0]}
- {id: 3, coords: [2.0, 0.0]}
- {id: 4, coords: [0.0, 0.5]}
- {id: 5, coords: [1.0, 0.5]}
- {id: 6, coords: [2.0, 0.5]}

thermal_materials:
- {id: 1, type: ThermalConduction, k: 45.0, c: 460.0, density: 7850.0}

elements:
- {id: 1, type: Quad4Thermal, nodes: [1, 2, 5, 4], material: 1, thickness: 0.3}
- {id: 2, type: Quad4Thermal, nodes: [2, 3, 6, 5], material: 1, thickness: 0.3}

boundary_conditions:
- {node_id: 1, T: 100.0}
- {node_id: 4, T: 100.0}
- {node_id: 3, T: 20.0}
- {node_id: 6, T: 20.0}

solver:
type: ThetaMethodSolver
dt: 2000.0
n_steps: 200
T_initial: 20.0

```

Con estos valores el nodo central llega a 59.998, frente al 60 exacto del perfil lineal estacionario. La diferencia **no es error del esquema**: con $\Delta t = 2000$ s y 200 pasos el análisis cubre 4×10^5 s, mientras el tiempo de difusión global de esta pared es $L^2/\alpha \approx 3.2 \times 10^5$ s — el transitorio está terminando pero aún no ha terminado. Es exactamente el efecto descrito en «Elección de Δt »: alargando el análisis, el campo converge al perfil lineal $100 \rightarrow 20$ que da el `LinearSolver` sobre el mismo modelo, a precisión de máquina.

Como la condición inicial (20 °C) contradice el Dirichlet impuesto (100 °C) en la cara izquierda, el solver **avisa** de la discontinuidad en $t = 0$ sin abortar: un choque térmico es modelización legítima, y es precisamente el caso que motiva el default $\theta = 1$.

8.8 Limitaciones actuales

- **Sin convección ni radiación.** Una frontera con convección $q = h(T - T_\infty)$ debe modelarse hoy como flujo prescrito equivalente, lo que exige conocer T de antemano — es decir, no es un sustituto general.
- **Sin acoplamiento termomecánico:** el campo de temperatura no produce deformación. Un análisis termoelástico requiere hoy resolver el térmico y trasladar el resultado a mano.
- **Material lineal:** k no depende de la temperatura, y no hay cambio de fase. Cualquiera de las dos cosas exigiría iteraciones de Newton dentro de cada paso temporal.
- **Paso de tiempo constante:** no hay adaptación automática.
- **Sólo elementos lineales:** Quad4Thermal y Hex8Thermal. No existen versiones cuadráticas ni triangulares/tetraédricas térmicas.

Capítulo 9

Post-Procesamiento y Visualización

9.1 Formato VTU (VTK XML)

Mediante `meshio`, Solidum FEM exporta los resultados en archivos `.vtu` compatibles con ParaView. La información se clasifica en dos diccionarios estándar:

9.1.1 Datos Nodales (Point Data)

- **Displacements:** campo vectorial (u_x, u_y, u_z) por nodo.
- **External_Forces:** distribución nodal del vector de cargas aplicado en el paso actual.
- **Supports:** indicador booleano de DOFs restringidos.

9.1.2 Datos de Elemento (Cell Data)

- **Von_Mises:** esfuerzo equivalente J2, calculado como invariante 3D con las seis componentes; en plane strain incluye σ_{zz} (que aporta el material vía `out_of_plane_stress`), en plane stress $\sigma_{zz} = 0$.
- **Internal_State:** variable principal del material según su `PRIMARY_STATE_VAR` (acumulada equivalente α en plasticidad, d en daño, etc.).
- **Sigma_XX, Sigma_YY, Sigma_ZZ, Tau_XY, Tau_YZ, Tau_XZ:** las seis componentes del tensor de esfuerzos (en 2D las tres últimas son σ_{zz} del material y ceros). Se evalúan en el campo `U` exportado vía `compute_gauss_state` y se promedian por elemento: el exportador no depende de qué solver produjo `U` ni de que el estado esté comiteado. Cubre sólidos 2D y 3D (`quad`, `triangle`, `quad8`, `quad9`, `triangle6`, `hexahedron`, `tetra`, `hexahedron20`, `hexahedron27`, `tetra10`), elementos térmicos (campo nodal `Temperature` y `Flux` por celda) y `CST_Embedded2D`.

9.2 Animación con Archivo PVD

Cuando se exporta con `frequency: "all_steps"`, además de un `.vtu` por paso se genera un archivo maestro `.pvd` (XML colección) que ParaView abre como animación temporal. El "tiempo" en el archivo PVD coincide con el factor de carga λ .

9.3 Salida en Texto Plano (Estilo FEAP)

Opcional para verificación manual o post-procesamiento con scripts. Configurable por nodos y elementos seleccionados:

```
output:
  text_export:
    export: true
    nodes: [1, 5, 10]           # o "all"
    elements: all
    nodal_results: ["Displacements", "External_Forces"]
    element_results: ["Von_Mises", "Internal_State"]
```

9.4 Workflow ParaView

1. File > Open y seleccionar el archivo .pvd (o el grupo de .vtu).
2. *Apply* en el panel *Properties*.
3. Reproducir la animación con los controles temporales de la barra superior.
4. Aplicar filtro **Warp By Vector** con **Displacements** para visualizar la deformación física (ajustar **Scale Factor** para amplificar).
5. Cambiar el coloreado de **Solid Color** a la variable de interés (**Von_Mises**, **Internal_State**, **Sigma_XX**, ...).

Capítulo 10

Ejemplos Completos

Nota

Este capítulo muestra la sintaxis de modelos completos. Los **problemas resueltos y verificados contra solución analítica**, con la explicación de cada paso, la lectura de resultados y la discusión de los errores, están en el *Manual de Ejemplos* ([manuals/Example_manual.pdf](#)).

10.1 Marco 2D — Viga en Voladizo

Combinación de un tramo Euler-Bernoulli y un tramo Timoshenko, empotrada en el extremo y cargada en la punta.

```
nodes:
- {id: 1, coords: [0.0, 0.0]}
- {id: 2, coords: [2.0, 0.0]}
- {id: 3, coords: [4.0, 0.0]}
```

```
materials:
- id: 1
  type: Elastic1D
  E: 200.0e9
  density: 7850.0
```

```
elements:
- id: 1
  type: Frame2DEuler
  material: 1
  nodes: [1, 2]
  A: 0.01
  I: 8.33e-6

- id: 2
  type: Frame2DTimoshenko
  material: 1
  nodes: [2, 3]
  A: 0.01
  I: 8.33e-6
  As: 0.00833
```

```

boundary_conditions_by_node:
  - {node_id: 1, ux: 0.0, uy: 0.0, rz: 0.0}

point_loads_by_node:
  - {node_id: 3, uy: -15000.0}

solver:
  type: LinearSolver

output:
  file_name: "viga_voladizo_marco"
  pre_process: {export: true}
  results:
    frequency: "last_step"
    nodal_results: ["Displacements"]

```

10.2 Placa Continua con Plasticidad J2 (Gmsh)

Placa con perforación central importada desde .msh, sometida a carga axial creciente hasta superar el límite elástico del acero. Solver no lineal con paso adaptativo.

```

mesh: "placa_agujero.msh"

mesh_physical_groups:
  "Superficie_Placa":
    material: 1
    thickness: 0.05
    quadrature: "2x2"

materials:
  - id: 1
    type: VonMises2D
    E: 200.0e9
    nu: 0.3
    sigma_y: 250.0e6
    H: 1.5e9
    hypothesis: plane_strain
    density: 7850.0

boundary_conditions_by_group:
  "Borde_Fijo": {ux: 0.0, uy: 0.0}

point_loads_by_group:
  "Borde_Carga": {ux: 1500000.0}

solver:
  type: NonlinearSolver
  num_steps: 30
  max_iter: 10
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5

```

```

output:
  file_name: "estudio_placa"
  pre_process: {export: true}
  results:
    frequency: "all_steps"
    nodal_results: ["Displacements"]
    element_results: ["Von_Mises", "Internal_State"]

```

Interpretación: durante los primeros pasos (régimen elástico) Newton-Raphson converge en 1–2 iteraciones. Cuando la concentración de esfuerzos en la periferia del orificio supera $\sigma_y = 250$ MPa, comienzan iteraciones del *return mapping*; **Internal_State** (la deformación plástica acumulada α) crece visiblemente alrededor del agujero.

10.3 Cable Pretensado en Régimen Corotacional

Cable 2D con material unilateral, cargado transversalmente en su nodo central. Demuestra el acoplamiento elemento corotacional + material unilateral.

```

nodes:
  - {id: 1, coords: [0.0, 0.0]}
  - {id: 2, coords: [5.0, 0.0]}
  - {id: 3, coords: [10.0, 0.0]}

materials:
  - id: 1
    type: CableMaterial1D
    E: 150.0e9
    density: 7850.0

elements:
  - id: 1
    type: Cable2DCorot
    material: 1
    nodes: [1, 2]
    A: 5.0e-5

  - id: 2
    type: Cable2DCorot
    material: 1
    nodes: [2, 3]
    A: 5.0e-5

boundary_conditions_by_node:
  - {node_id: 1, ux: 0.0, uy: 0.0}
  - {node_id: 3, ux: 0.0, uy: 0.0}

point_loads_by_node:
  - {node_id: 2, uy: -500.0}

solver:
  type: NonlinearSolver
  num_steps: 50
  max_iter: 25

```



```
adaptive: true
convergence:
  rtol_force: 1.0e-5
  rtol_disp: 1.0e-5

output:
  file_name: "cable_carga_transversal"
  pre_process: {export: true}
  results:
    frequency: "all_steps"
    nodal_results: ["Displacements"]
```

Capítulo 11

Uso Avanzado: API Programática

Aunque el flujo principal es *data-driven* (YAML), el motor se puede usar como librería Python para escenarios donde el archivo no es suficiente: optimización paramétrica, generación procedural de mallas, acoplamiento con bibliotecas externas, scripting de batería de tests.

11.1 Patrón Fachada

Los componentes principales se exponen desde la raíz del paquete:

```
import numpy as np
from solidum import Domain, VonMises2D, Quad4, Assembler, NonlinearSolver, VtkExporter

# 1. Dominio y nodos
domain = Domain()
n1 = domain.add_node(1, [0.0, 0.0])
n2 = domain.add_node(2, [1.0, 0.0])
n3 = domain.add_node(3, [1.0, 1.0])
n4 = domain.add_node(4, [0.0, 1.0])

# 2. Material y elemento
acero = VonMises2D(E=200e9, nu=0.3, sigma_y=250e6, H=2e9, hypothesis='plane_strain')
placa = Quad4(element_id=1, nodes=[n1, n2, n3, n4], material=acero, thickness=0.1)
domain.add_element(placa)

# 3. Restricciones (Dirichlet)
n1.fix_dof('ux', 0.0); n1.fix_dof('uy', 0.0)
n4.fix_dof('ux', 0.0); n4.fix_dof('uy', 0.0)

domain.generate_equation_numbers()

# 4. Cargas (Neumann)
F_ext = np.zeros(domain.total_dofs)
F_ext[n2.dofs['ux']] = 150000.0
F_ext[n3.dofs['ux']] = 150000.0

# 5. Resolución
assembler = Assembler(domain)
solver = NonlinearSolver(assembler, num_steps=10, adaptive=True)
U = solver.solve(F_ext)
```

```
# 6. Exportación
VtkExporter(domain).export("resultados.vtu", U=U, F_ext=F_ext)
```

`Assembler(domain)` ensambla **por lotes** (ADR 0014): agrupa los elementos que comparten clase, material y cuadratura en familias y los evalúa en un kernel compilado, con el mismo resultado que el bucle elemento a elemento a precisión de máquina. `Assembler(domain, batch=False)` fuerza el camino por elemento en todo el modelo (útil para comparar o diagnosticar); `parallel=False` usa el kernel serie en vez del paralelo (el resultado es bit a bit el mismo; el número de hilos lo fija Numba con la variable de entorno `NUMBA_NUM_THREADS` o `numba.set_num_threads`); `batch_memory_budget=` acota, en bytes, los temporales por trozo. Los elementos sin kernel por lotes (1D, discontinuidad embebida) se ensamblan como siempre dentro del mismo `Assembler`. Si se sustituye el material o el espesor de un elemento después del primer ensamblaje, llamar a `assembler.invalidate()`. Un elemento con nodos en orden incorrecto (jacobiano negativo) detiene el ensamblaje con `ValueError` que nombra su id.

Para obtener deformaciones y esfuerzos en los puntos de Gauss de todo el modelo, `solidum.math.batch.gauss_states(domain, U)` devuelve `{id: compute_gauss_state(U)}` evaluando cada familia de una vez (las entradas son vistas sobre arreglos) y llamando a `compute_gauss_state` sólo en los elementos sin familia; el exportador VTK hace lo mismo por dentro.

El exportador evalúa los esfuerzos en el `U` que recibe (`compute_gauss_state`), así que funciona igual tras `LinearSolver.solve` directo, tras `solidum.run` o en un paso intermedio del `step_callback`. Si el modelo lleva peso propio o fuerza de cuerpo, usar `solidum.run(...)` en vez de `solver.solve(...)` para que `SolveResult.element_forces` reste la carga nodal equivalente y devuelva fuerzas internas de extremo coherentes con las reacciones.

11.2 Errores tipados

Desde un script conviene capturar los errores por su tipo, porque cada uno indica una causa y una salida distintas:

```
from solidum.math.solvers.diagnostics import (
    MechanismError,          # el modelo no está suficientemente apoyado (antes de
                             resolver)
    IllPosedSystemError,     # solución no fiable: mecanismo interno o matriz singular
    SolverDivergedError,     # familia del Newton: oscilación, tangente singular,
                             capacidad...
)
from solidum.math.linalg import IterativeNotConvergedError # sólo con linear_algebra:
    iterative

try:
    U = solver.solve(F_ext)
except MechanismError as err:
    print(err.motions)      # p. ej. ['traslación en la dirección x']
```

`MechanismError.motions` lista los movimientos libres ya descritos en palabras. Los mensajes completos y cómo corregir cada caso están en el capítulo *Diagnóstico de Problemas*.

El solver del sistema lineal se puede fijar también desde Python con el mismo parámetro que en el YAML, p. ej. `LinearSolver(assembler, linear_algebra="iterative")`.

Capítulo 12

Diagnóstico de Problemas

- `YamlValidationError`: parámetro 'X' no aceptado por 'Y'.

El parser detecta un parámetro que el constructor del elemento no admite. La salida lista los parámetros aceptados; revisar el catálogo o la spec del componente.

- `MechanismError`: El modelo no está suficientemente apoyado para un análisis estático...

Los apoyos no impiden que el modelo se mueva como sólido rígido, así que el problema estático no tiene solución única. Solidum lo detecta **antes de resolver** y dice exactamente qué movimiento queda libre, por ejemplo *"traslación en la dirección x"* o *"giro alrededor del eje z que pasa por (0, 0.5)"*. Añadir apoyos (o restricciones lineales) que impidan esos movimientos. Ojo con los rodillos alineados: impedir solo *uy* a lo largo de una recta vertical deja libre el giro alrededor de cualquier punto de esa recta. En un modelo térmico estacionario, el mensaje equivalente es que el campo **T** no tiene ningún valor prescrito. En análisis modal o dinámico un modelo libre sí es válido y no se comprueba.

- `IllPosedSystemError`: Resultado descartado...

El modelo está bien apoyado en conjunto, pero la solución no es fiable: o no satisface el equilibrio, o la matriz tiene pivotes numéricamente nulos. Casi siempre es un **mecanismo interno**: una rótula o articulación de más, una barra o elemento suelto, o dos partes unidas por un solo nodo que pueden girar entre sí. Más raramente, rigideces muy desproporcionadas entre partes del modelo (penalizaciones, unidades mezcladas). Revisar la conectividad. Antes de esta comprobación, un modelo así devolvía desplazamientos absurdos sin ningún aviso.

- `SingularTangentError` en un análisis no lineal

La rigidez tangente perdió rango: un punto límite o bifurcación (usar `ArcLengthSolver`), o un mecanismo (ver los dos puntos anteriores). Si el mensaje añade *"el sistema tangente no se pudo resolver con precisión"*, la causa más probable es un mecanismo.

- `MemoryError`: Memoria insuficiente para factorizar...

El modelo es demasiado grande para el solver directo, cuya memoria crece más deprisa que el modelo. Usar `linear_algebra: iterative` en el bloque `solver` (ver el anexo de la capa algebraica).

- **Cables y materiales con ablandamiento.**

En modelos con cables o materiales con softening pronunciado, un elemento que se destensa o se degrada puede dejar un mecanismo: comprobar que existen caminos de carga alternativos.

- **Newton-Raphson no convergió / biseando incremento.**

El paso de carga es demasiado grande. Aumentar `num_steps`, asegurar `adaptive: true`. Verificar tipo de los parámetros constitutivos (σ_y , κ_0 , etc.). Si el problema tiene snap-through o softening, cambiar a `ArcLengthSolver`.

- **No se importó ningún elemento (Solid2D).**

El parser de Gmsh no encontró superficies bidimensionales. En Gmsh, crear una *Plane Surface* y generar la malla 2D antes de exportar.

- **Cable totalmente destensado --- $KT = 0$.**

Pretensar el cable o garantizar que la estructura tiene rigidez residual por otros elementos. Considerar también pasos de carga finos para capturar la transición tensado $\rightarrow$ destensado.

Solidum FEM — Manual de Usuario

Sigla del manual: FF-MU

Compilado el 23 de septiembre de 2026, 11:47

Commit Git: b8d73af

*Documento generado automáticamente desde los archivos fuente en
manuals/sources/user/ mediante `python manuals/build_user_manual.py`.
No editar directamente el PDF: cualquier corrección debe realizarse sobre el
archivo fuente correspondiente y regenerar el manual.*